

Module 3

Differential Equations

3.1 Numerical Solution of Ordinary Differential Equations

3.1.1 Some Pertinent Concepts and Definitions

An n th order differential equation may be written in the general form

$$F\left(x, y, \frac{dy}{dx}, \frac{d^2y}{dx^2}, \dots, \frac{d^ny}{dx^n}\right) = 0 \quad (3.1)$$

Eq. (3.1) is termed n th order because the highest derivative appearing in the equation is of order n and *ordinary* because only total derivatives are present (there is only one independent variable).

A function $y(x)$ that *satisfies* Eq. (3.1) is said to be a *solution* of the equation. This means that if $y(x)$, $dy(x)/dx$, $\dots$, $d^ny(x)/dx^n$ are evaluated at any given x and substituted in (3.1), the result is $0 = 0$. In general, there is an infinite number of functions $y(x)$ which satisfy (3.1). To obtain a *unique solution*, it is necessary to supply additional information, usually values of $y(x)$ and/or of its derivatives at some specific value, or values, of x . For an n th order ODE, n such *conditions* are normally sufficient to yield a unique solution.

If all n conditions are specified at only one value of x , usually denoted x_0 , the problem is called an *initial-value problem* (IVP); if the n conditions are specified at more than one value of x , the problem is called a *boundary-value problem* (BVP). BVPs are generally more difficult to solve than IVPs.

Example 1: Consider the first order ODE

$$\frac{dy}{dx} = y$$

The analytical solution is

$$\begin{aligned} \int \frac{dy}{y} &= \int dx \\ \ln y &= x + \ln c \\ \ln \frac{y}{c} &= x \end{aligned}$$

or, finally

$$y(x) = ce^x$$

which is a family of curves for different values of c . If the condition $y(0) = 1$ is given, we obtain $c = 1$ and the unique solution $y(x) = e^x$ (the middle curve in the figure).

Example 2: The 2nd order ODE

$$\frac{d^2\theta}{dt^2} + \frac{g}{L} \sin \theta = 0$$

describes the motion of a swinging pendulum. This ODE has *no* analytical solution $\theta(t)$ in terms of elementary functions. This is typical of many ODEs which arise in the solution to problems in engineering, physics, chemistry and other fields. They are either extremely difficult or impossible to solve analytically. In such cases, numerical methods offer the only viable alternative.

3.1.2 The Generic Problem in ODEs

Any n th order ODE can be written as a system of n first order ODEs by defining $n - 1$ new variables. Consider, for example, the second order ODE

$$x^2 \frac{d^2y}{dx^2} + x \frac{dy}{dx} + (x^2 - p^2)y = 0$$

Letting $z = dy/dx$, we can rewrite the equation as a system of two first order ODEs, thus:

$$\begin{aligned} \frac{dy}{dx} - z &= 0 \\ x^2 \frac{dz}{dx} + xz + (x^2 - p^2)y &= 0 \end{aligned}$$

It is clear that a *system* of high order ODEs can be similarly reduced to an equivalent system of first order ODEs.

Thus the generic problem in ODEs becomes that of solving a system of n coupled first order ODEs for the functions $y_i(x)$, $i = 1, 2, 3, \dots, n$, having the general form

$$\begin{aligned} \frac{dy_1(x)}{dx} &= f_1(x, y_1, y_2, y_3, \dots, y_n) \\ \frac{dy_2(x)}{dx} &= f_2(x, y_1, y_2, y_3, \dots, y_n) \\ \frac{dy_3(x)}{dx} &= f_3(x, y_1, y_2, y_3, \dots, y_n) \\ &\vdots \\ \frac{dy_n(x)}{dx} &= f_n(x, y_1, y_2, y_3, \dots, y_n) \end{aligned} \tag{3.2}$$

or, concisely

$$\frac{dy_i(x)}{dx} = f_i(x, y_1, y_2, y_3, \dots, y_n) \quad i = 1, 2, \dots, n \tag{3.2}$$

where the functions f_i on the right-hand side are known. To obtain a unique solution to Eqs. (3.2), n initial or boundary conditions must be specified.

Example: The figure shows a horizontally launched, freely falling body subjected to air resistance. Its equations of motion are given by the system of 2nd order ODEs

$$\begin{aligned} \frac{d^2x}{dt^2} &= -\frac{g}{V_t^2} \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2} \frac{dx}{dt} \\ \frac{d^2y}{dt^2} &= g - \frac{g}{V_t^2} \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2} \frac{dy}{dt} \end{aligned}$$

where g is the acceleration due to gravity and V_t is the terminal velocity = 160 ft/s. Letting $u = dx/dt$ and $w = dy/dt$, the given ODEs can be written as a system of four first order ODEs

$$\begin{aligned}\frac{dx}{dt} &= u & &= f_1(t, x, y, u, w) \\ \frac{dy}{dt} &= w & &= f_2(t, x, y, u, w) \\ \frac{du}{dt} &= -\frac{g}{v_t^2} \sqrt{u^2 + w^2} u & &= f_3(t, x, y, u, w) \\ \frac{dw}{dt} &= g - \frac{g}{v_t^2} \sqrt{u^2 + w^2} w & &= f_4(t, x, y, u, w)\end{aligned}$$

with initial conditions

$$\left. \begin{aligned}x(0) &= 0 \\ y(0) &= 0\end{aligned} \right\} \quad \text{point of launch is the origin}$$

$$\begin{aligned}u(0) &= 440 \text{ ft/s} &= \text{given initial horizontal velocity} \\ w(0) &= 0 &\text{since body is launched horizontally}\end{aligned}$$

3.2 First Order Ordinary Differential Equations

Since any high order ODE, or a system of such ODEs, can be recast as an equivalent system of first order ODEs, it suffices to develop methods of solving a generic first order ODE. These methods can subsequently be applied to find the solution to a system of first order ODEs.

Our problem, then, is to find a solution to the ODE

$$\frac{dx}{dy} = f(x, y) \quad (3.3)$$

subject to the initial condition

$$y(x_0) = y_0 \quad (3.4)$$

Analytically, the solution to this problem is a function $y(x)$ which satisfies both the given ODE (3.3) and initial condition (3.4). We will refer to $y(x)$ as the *solution*, or *object function*. In general, unless the function $f(x, y)$ in (3.3) is of a 'special' form, it is rather difficult, if not impossible, to find $y(x)$.

Numerically, the solution to (3.3) subject to (3.4) is a tabulation of computed y -values vs specified x -values, as shown.

x_0	y_0	← Initial condition
x_1	y_1	
x_1	y_1	
$\vdots$	$\vdots$	
x_i	y_i	
x_{i+1}	y_{i+1}	
$\vdots$	$\vdots$	
x_n	y_n	

For now, let us take note of the following important points about the numerical solution given above:

1. Unlike the analytical solution $y(x)$ which (if known) yields the value of y for *any* value of x , the numerical solution gives the value of y only for *discrete* values of x over a specified, finite interval $[x_0, x_n]$, called the *solution interval*.
2. The solution interval is divided into subintervals called *steps*; the size of a subinterval (i.e., the distance between two successive values of x) is called the *stepsize*. The stepsize plays a fundamental role in the solution to the ODE; it can be handled in one of two ways:

- (a) a fixed value of the stepsize is specified and is used throughout the solution interval;
 - (b) an error tolerance is specified and the method adjusts the stepsize, every step of the way, such that the specified error tolerance is satisfied (this technique is referred to as *adaptive stepsize control*).
3. The solution table is generated row by row, starting from the given initial condition. To generate the next row is to perform an *integration step*. An integration step is defined by the basic scheme

$$y_{i+1} = y_i + \Delta y.$$

The primary difference among the different numerical methods for solving ODEs lies in the way the increment Δy is determined.

$[x_0, x_n]$	=	interval in x over which a solution is desired
n	=	number of subintervals, or <i>steps</i> , in $[x_0, x_n]$
h	=	$\frac{x_n - x_0}{n}$ = <i>step size</i>
x_i	=	$x_0 + ih, i = 1, 2, \dots, n$ = specified values of x at which $y(x)$ is computed
$y(x)$	=	solution, or object, function (not known)
$y(x_i)$	=	true value of $y(x)$ at x_i (not known)
y_i	=	computed approximation of $y(x)$ at x_i
$f(x_i, y(x_i))$	=	true value of dy/dx at x_i (not known)
$f(x_i, y_i)$	=	computed approximation of dy/dx at x_i
ε_i	=	$y_i - y(x_i)$ = truncation error (assuming no round-off error)

3.3 Methods for Solving First Order ODEs

We wish to solve the IVP

$$\frac{dy}{dx} = f(x, y) \quad \text{such that} \quad y(x_0) = y_0 \quad (3.5)$$

There are four general categories of methods for numerically solving this problem.

1. **Taylor series method**—this method is based on the direct use of the Taylor series expansion (TSE) of the object function $y(x)$ about $x = x_i$ to obtain $y(x)$ at $x = x_i$. An integration step in the method is thus given by

$$\begin{aligned} y(x_{i+1}) &= y(x_i + h) \\ &= y(x_i) + \frac{h}{1!}y'(x_i) + \frac{h^2}{2!}y''(x_i) + \frac{h^3}{3!}y'''(x_i) + \dots \\ &= y(x_i) + \frac{h}{1!}f(x_i, y(x_i)) + \frac{h^2}{2!}f'(x_i, y(x_i)) + \frac{h^3}{3!}f''(x_i, y(x_i)) + \dots \end{aligned} \quad (3.6)$$

Since it is not possible to sum an infinite series of terms, we have to *truncate* the series, for instance

$$\begin{aligned} y_{i+1} &= y(x_i + h) \\ &= y_i + \frac{h}{1!}f(x_i, y_i) + \frac{h^2}{2!}f'(x_i, y_i) + \frac{h^3}{3!}f''(x_i, y_i) + \frac{h^4}{4!}f'''(x_i, y_i) \end{aligned} \quad (3.7)$$

In obtaining (3.7) from (3.6), we say that we have *retained terms through h^4* . Note that the consequent change in notation from 'exact' to 'approximate'. The derivatives f' , f'' , f''' , etc. are obtained by applying the chain rule.

The effectiveness of the Taylor series method when implemented on a computer depends on the availability of software which can compute the derivatives f', f'', f''' , etc. of an arbitrary function $f(x, y)$. If one has no access to, or is not inclined to write, such software, then one has to generate the derivatives 'by hand' and write the corresponding code. This makes the *direct* use of the Taylor series of limited practical applicability.

This notwithstanding, it is extremely important that you understand what the Taylor series is all about if you are to understand the inner workings of the other methods.

2. **Runge-Kutta methods**—these methods are designed to approximate the Taylor series method *without requiring the evaluation of the derivatives of $f(x, y)$* . In lieu of evaluating f', f'', f''' , etc., f is instead evaluated several times.

An integration step in the Runge-Kutta methods is given by the formula

$$y_{i+1} = y_i + h \cdot \phi(x_i, y_i, h)$$

where the *increment function* $\phi(x_i, y_i, h)$ is simply a suitably chosen approximation to the function $f(x, y)$ on the integration interval $[x_i, x_{i+1}]$. The R-K methods are *single-step* methods, which means that only the information at the point (x_i, y_i) is needed to perform an integration step. Two other properties immediately follow from this: (a) the methods are self-starting; and (b) the step size can be changed at any step in the computations. The Runge-Kutta methods are easily implemented on a computer.

3. **Predictor-corrector methods**—these methods differ from the Runge-Kutta methods in that to perform an integration step they require information not only at the point (x_i, y_i) on the current integration interval $[x_i, x_{i+1}]$, but also at previous points (x_{i-1}, y_{i-1}) , (x_{i-2}, y_{i-2}) , (x_{i-3}, y_{i-3}) , etc. As such, they are *multistep* methods. As an example, consider Hamming's 4th-order predictor-corrector method:

$$\text{P:} \quad y_{i+1} = y_{i-3} + \frac{4}{3}h [2f(x_i, y_i) - f(x_{i-1}, y_{i-1}) + 2f(x_{i-2}, y_{i-2})]$$

$$\text{C:} \quad y_{i+1} = \frac{9}{8}y_i - \frac{1}{8}y_{i-2} + \frac{3}{8}h [f(x_{i+1}, y_{i+1}) + 2f(x_i, y_i) - f(x_{i-1}, y_{i-1})]$$

Predictor-corrector methods usually require less computational effort than the Runge-Kutta methods for the same order of accuracy. However, they are not self-starting, and the step size is not readily changed as the solution progresses. They generally require more attention to programming details when implemented on a computer.

4. **Methods based on Richardson extrapolation**—a representative method in this category is the Bulirsch-Stoer method. (These methods are not covered in this 3rd Module; the interested student is referred to Press, W., *et al*, "Numerical Methods in C: The Art of Scientific Computing", Cambridge University Press, 1992.)

3.4 Taylor Series Method

Given the IVP

$$\frac{dy}{dx} = f(x, y) \quad \text{such that} \quad y(x_0) = y_0$$

one method of approximating the solution is to expand the solution function $y(x)$ in a Taylor series about $x = x_i$ to obtain $y(x)$ at $x = x_i + h = x_{i+1}$:

$$\begin{aligned}
y(x_{i+1}) &= y(x_i + h) \\
&= y(x_i) + \frac{h}{1!}f(x_i, y(x_i)) + \frac{h^2}{2!}f'(x_i, y(x_i)) + \frac{h^3}{3!}f''(x_i, y(x_i)) + \cdots + \\
&\quad \frac{h^n}{n!}f^{(n-1)}(x_i, y(x_i)) + \frac{h^{n+1}}{(n+1)!}f^{(n)}(\xi, y(\xi)) \quad x_i \leq \xi \leq x_{i+1}
\end{aligned} \tag{3.6}$$

The higher derivatives in (3.6) are obtained by using the chain rule, f being in general a function of x and y .

$$\begin{aligned}
\frac{dy}{dx} &= f(x, y) \equiv f \quad (\text{given ODE}) \\
\frac{df}{dx} &= \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} \frac{dy}{dx} = \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f \equiv f' \\
\frac{d^2 f}{dx^2} &= \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f \right) + \frac{\partial}{\partial y} \left(\frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f \right) \equiv f''
\end{aligned}$$

Example 1

$$\frac{dy}{dx} = f(x, y) = x^2 \quad \text{such that} \quad y(x_0) = y_0$$

$$\begin{aligned}
f(x, y) &= x^2 \quad (\text{given function}) \\
f'(x, y) &= 2x \\
f''(x, y) &= 2 \\
f'''(x, y) &= 0 = f''''(x, y) = \dots
\end{aligned}$$

$$\begin{aligned}
y(x_{i+1}) &= y(x_i) + \frac{h}{1!}f(x_i, y(x_i)) + \frac{h^2}{2!}f'(x_i, y(x_i)) + \frac{h^3}{3!}f''(x_i, y(x_i)) + \cdots \\
&= y(x_i) + \frac{h}{1!}x_i^2 + \frac{h^2}{2!}(2x_i) + \frac{h^3}{3!}(2) \\
&= y(x_i) + hx_i^2 + h^2x_i + \frac{1}{3}h^3
\end{aligned} \tag{3.8}$$

Starting from the given initial condition, the solution table is generated row by row by repeated application of Eq. (3.8). Note that since the 3rd and higher derivatives vanish, the values given by (3.8) are exact (ignoring round-off errors).

$$\begin{aligned}
y(x_1) &= y(x_0) + hx_0^2 + h^2x_0 + \frac{1}{3}h^3 \\
y(x_2) &= y(x_1) + hx_1^2 + h^2x_1 + \frac{1}{3}h^3 \\
y(x_3) &= y(x_2) + hx_2^2 + h^2x_2 + \frac{1}{3}h^3 \\
&\vdots
\end{aligned}$$

Example 2

$$\frac{dy}{dx} = f(x, y) = 2y \quad \text{such that} \quad y(x_0) = y_0$$

$$\begin{aligned}
f(x, y) &= 2y \quad (\text{given function}) \\
f'(x, y) &= \frac{\partial f}{\partial x} + \frac{\partial f}{\partial y} f = 0 + 2(2y) = 4y
\end{aligned}$$

$$\begin{aligned}
f''(x, y) &= \frac{\partial f'}{\partial x} + \frac{\partial f'}{\partial y} f = 0 + 4(2y) = 8y \\
f'''(x, y) &= \frac{\partial f''}{\partial x} + \frac{\partial f''}{\partial y} f = 0 + 8(2y) = 16y \\
&\vdots
\end{aligned}$$

$$y(x_{i+1}) = y(x_i) + \frac{h}{1!} f(x_i, y(x_i)) + \frac{h^2}{2!} f'(x_i, y(x_i)) + \frac{h^3}{3!} f''(x_i, y(x_i)) + \frac{h^4}{4!} f'''(x_i, y(x_i)) + O(h^5)$$

Retaining terms through h^4 and substituting the expressions for the derivatives, we obtain

$$\begin{aligned}
y_{i+1} &= y_i + h(2y_i) + \frac{h^2}{2!}(4y_i) + \frac{h^3}{3!}(8y_i) + \frac{h^4}{4!}(16y_i) \\
&= y_i \left(1 + 2h + \frac{(2h)^2}{2!} + \frac{(2h)^3}{3!} + \frac{(2h)^4}{4!} \right) \\
&= y_i \alpha
\end{aligned} \tag{3.9}$$

Starting from the given initial condition, the solution table is generated row by row by repeated application of Eq. (3.9).

$$\begin{aligned}
y_1 &= y_0 \alpha \\
y_2 &= y_1 \alpha \\
y_3 &= y_2 \alpha \\
&\vdots \\
y_{i+1} &= y_i \alpha \\
&\vdots
\end{aligned} \tag{3.9}$$

Note that the factor α in Eq. (3.9) is approximate since only five out of an infinite number of terms have been retained; this implies that Eq. (3.9) is also approximate. Thus, *ignoring round-off error*, there are two kinds of error which arise when Eq. (3.9) is applied to perform an integration step.

- (a) **local truncation error**—this is the error in y_{i+1} due to the fact that Eq. (3.9) is approximate, *assuming that y_i is exact*.
- (b) **propagation error**—this is the error in y_{i+1} due to the fact that y_i is in fact *not exact* (with the exception of y_0).

The total error in y_{i+1} is called the *global truncation error* and is the sum of the local truncation error and the propagation error.

Now assume that we are given the initial condition $y(0) = 1$. The following tables show the effect of α (i.e., the number of terms retained in the Taylor series) and the stepsize h on the numerical solution on the interval $[0, 3]$. The exact solution is $y(x) = e^{2x}$ as you may easily verify:

$$y(1.0) = 7.389056 \quad y(2.0) = 54.598150 \quad y(3.0) = 403.428793$$

$\alpha = 1 + 2h$								
x_i	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$
	$h = 1.0$		$h = 0.1$		$h = 0.01$		$h = 0.001$	
1.0	3.000000	-4.389056	6.191736	-1.197320	7.244646	-0.144410	7.374312	-0.014744
2.0	9.000000	-45.598150	38.337600	-16.260550	52.484897	-2.113253	54.380483	-0.217667
3.0	27.000000	-376.428793	237.376314	-166.052480	380.234508	-23.194285	401.018671	-2.410122

$\alpha = 1 + 2h + \frac{(2h)^2}{2!}$								
x_i	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$
	$h = 1.0$		$h = 0.1$		$h = 0.01$		$h = 0.001$	
1.0	5.000000	-2.389056	7.304631	-0.084425	7.388086	-0.000970	7.389046	-0.000010
2.0	25.000000	-29.598150	53.357640	-1.240510	54.583809	-0.014341	54.598005	-0.000145
3.0	125.000000	-278.428793	389.757894	-13.670899	403.269855	-0.158939	403.427182	-0.001611

$\alpha = 1 + 2h + \frac{(2h)^2}{2!} + \frac{(2h)^3}{3!}$								
x_i	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$
	$h = 1.0$		$h = 0.1$		$h = 0.01$		$h = 0.001$	
1.0	6.333333	-1.055723	7.384857	-0.004199	7.389051	-0.000005	7.389056	0.000000
2.0	40.111111	-14.487039	54.536116	-0.062034	54.598078	-0.000072	54.598150	0.000000
3.0	254.037037	-149.391756	402.741430	-0.687363	403.427999	-0.000794	403.428792	-0.000001

$\alpha = 1 + 2h + \frac{(2h)^2}{2!} + \frac{(2h)^3}{3!} + \frac{(2h)^4}{4!}$								
x_i	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$	y_i	$y_i - y(x_i)$
	$h = 1.0$		$h = 0.1$		$h = 0.01$		$h = 0.001$	
1.0	7.000000	-0.389056	7.388889	-0.000167	7.389056	0.000000	7.389056	0.000000
2.0	49.000000	-5.598150	54.595684	-0.002466	54.598150	0.000000	54.598150	0.000000
3.0	343.000000	-60.428793	403.401464	-0.027330	403.428790	-0.000003	403.428793	0.000000

3.5 The Euler Method

Except for a few toy problems, getting the higher order derivatives of the function $f(x, y)$ can become a very complicated affair. This makes the direct use of the TSE of $y(x)$ to solve first order ODEs of limited practical importance (unless we have software that performs symbolic differentiation of arbitrary functions), if we insist on retaining terms of the series through h^p , $p \geq 2$. However, if we are willing to drop all terms except the first two, we obtain a simple procedure called *Euler's method*.

$$y_{i+1} = y_i + h \cdot f(x_i, y_i) \quad (3.10)$$

Eq. (3.10) has a very simple geometrical interpretation, as indicated below.

Starting from the initial point $(x_0, y(x_0))$, Euler's method approximates the solution function $y(x)$ by a sequence of line segments with slopes $f(x_0, y(x_0)), f(x_i, y(x_i)), i = 1, 2, 3, \dots$

A measure of accuracy of a method for solving ODEs numerically is the so-called *order* of the method. In general, the higher the order of the method, the more accurate the results (for the same stepsize, of course). A method is said to be of order p if the local truncation error incurred in integrating over one step is $O(h^{p+1})$. Recall that we obtained Euler's method by dropping the terms in $O(h^2)$ from the TSE of $y(x)$, which means that the local truncation error incurred by the application of Eq. (3.10) is $O(h^2)$. Thus Euler's method is of order 1.

Another way of looking at the matter is in terms of the global truncation error in y_n . Assuming that the solution interval $s = [x_0, x_n]$ is divided into n subintervals of size h , the global truncation error incurred in applying Euler's method over n steps is $n \cdot O(h^2) = (s/h) \cdot O(h^2) = O(h^1)$, which is the order of Euler's method.

Problem: Shown is a simple electrical circuit consisting of a resistor R , an inductor L and a constant voltage source E . When the switch is closed, current flows through the circuit. The governing ODE is

$$L \frac{di}{dt} + Ri = E$$

subject to the initial condition $i(0) = 0$. Given that $E = 10$ volts, $R = 2$ ohms and $L = 5$ henries, find i at $t = 1$ sec, 2 sec, 3 sec and 4 sec after the switch is closed.

Solution: We write the ODE in canonical form, thus

$$\frac{di}{dt} = \frac{E - Ri}{L} = f(t, i)$$

Then, Euler's formula becomes

$$i_{j+1} = i_j + h \cdot f(t_j, i_j) = i_j + h \frac{E - Ri_j}{L} = i_j + h \frac{10 - 2i_j}{5}$$

For $h = 1$ sec, we obtain:

$$\begin{aligned} i_1 &= i_0 + h(2.0 - 0.4i_0) = 0 + 1[2.0 - 0.4(0)] = 2.00000 \\ i_2 &= i_1 + h(2.0 - 0.4i_1) = 2.00000 + 1[2.0 - 0.4(2.00000)] = 3.20000 \\ &\vdots \end{aligned}$$

The following tables summarize the results for $h = 1.0$ sec, 0.1 sec, 0.01 sec, 0.001 sec, 0.0001 sec and 0.00001 sec. The results are compared with the analytical solution

$$i(t) = \frac{E}{R}(1 - e^{-Rt/L}) = 5.0(1 - e^{-0.4t})$$

$h = 1.0$ sec				$h = 10^{-1}$ sec		$h = 10^{-2}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error	i_j	Error
1	1.648400	2.000000	0.351600D+00	1.675837	0.274370D-01	1.651087	0.268737D-02
2	2.753355	3.200000	0.446645D+00	2.789988	0.366326D-01	2.756957	0.360134D-02
3	3.494029	3.920000	0.425971D+00	3.530712	0.366828D-01	3.497649	0.361962D-02
4	3.990517	4.352000	0.361483D+00	4.023169	0.326518D-01	3.993751	0.323376D-02

$h = 10^{-3}$ sec				$h = 10^{-4}$ sec		$h = 10^{-5}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error	i_j	Error
1	1.648400	1.648668	0.268184D-03	1.648427	0.268088D-04	1.648402	0.267670D-05
2	2.753355	2.753715	0.359516D-03	2.753391	0.359327D-04	2.753359	0.358039D-05
3	3.494029	3.494390	0.361461D-03	3.494065	0.361187D-04	3.494033	0.358920D-05
4	3.990517	3.990840	0.323033D-03	3.990550	0.322684D-04	3.990521	0.319495D-05

Note that decreasing the stepsize by a factor 10^{-p} reduces the error by the same factor, i.e., the error decreases linearly with h . This follows from the fact that Euler's method is of order 1, which means (by definition of order) that the global truncation error is $O(h^1)$.

3.6 The Runge-Kutta Methods

3.6.1 Some General Properties of the Runge-Kutta Methods

1. They are one-step methods. In advancing the solution over the integration interval $[x_i, x_{i+1}]$, only the derivative information at (x_i, y_i) is needed.
2. They require the evaluation of $f(x, y)$ only and none of its derivatives.
3. Depending on the order of the method, they yield results equivalent in accuracy to the TSE of $y(x)$ with terms retained through h^2, h^3, h^4 , etc. For instance, a 4th order Runge-Kutta method yields results of the same order of accuracy as the Taylor series

$$y_{i+1} = y_i + \frac{h}{1!}f(x_i, y_i) + \frac{h^2}{2!}f'(x_i, y_i) + \frac{h^3}{3!}f''(x_i, y_i) + \frac{h^4}{4!}f'''(x_i, y_i) \quad (3.7)$$

Note that using (3.7) requires evaluating f, f', f'' and f''' at (x_i, y_i) , which can be extremely cumbersome. Using a 4th order R-K method requires evaluating f only (4 times).

4. All Runge-Kutta methods have the form

$$y_{i+1} = y_i + h \cdot \phi(x_i, y_i, h) \quad (3.11)$$

where $\phi(x_i, y_i, h)$ is called the *increment function*. This is simply a suitably chosen approximation to $f(x, y)$ on the interval $[x_i, x_{i+1}]$.

3.6.2 Derivation of the Second Order Runge-Kutta Methods

Let $\phi(x_i, y_i, h)$ be the weighted average of two first order derivative evaluations k_1 and k_2 on the interval $[x_i, x_{i+1}]$, i.e.,

$$\begin{aligned} \phi(x_i, y_i, h) &= ak_1 + bk_2 \\ k_1 &= f(x_i, y_i) \\ k_2 &= f(x_i + ph, y_i + qhk_1) \end{aligned}$$

where a, b, p and q are constants to be established later. Expanding k_2 about (x_i, y_i) in a Taylor series for a function of two variables, we obtain

$$\begin{aligned} k_2 &= f(x_i + ph, y_i + qhk_1) \\ &= f(x_i, y_i) + \frac{(ph)^1}{1!} \frac{\partial f(x_i, y_i)}{\partial x} + \frac{(qh k_1)^1}{1!} \frac{\partial f(x_i, y_i)}{\partial y} + O(h^2) \\ &\approx f(x_i, y_i) + ph f_x(x_i, y_i) + qh f(x_i, y_i) f_y(x_i, y_i) \end{aligned}$$

where $f_x \equiv \partial f / \partial x$, $f_y \equiv \partial f / \partial y$ and we have dropped the terms in $O(h^2)$. Substituting these expressions into the basic computational scheme of the Runge-Kutta methods [Eq. (3.11)], we get

$$\begin{aligned} y_{i+1} &= y_i + h \cdot \phi(x_i, y_i, h) \\ &= y_i + h(ak_1 + bk_2) \\ &= y_i + h[af(x_i, y_i) + b[f(x_i, y_i) + ph f_x(x_i, y_i) + qh f(x_i, y_i) f_y(x_i, y_i)]] \\ &= y_i + h[(a+b)f(x_i, y_i)] + h^2[bp f_x(x_i, y_i) + bq f(x_i, y_i) f_y(x_i, y_i)] \end{aligned} \quad (3.12)$$

Next, we expand the solution function $y(x)$ in a Taylor series, retaining terms through h^2 :

$$\begin{aligned} y_{i+1} &= y_i + hf(x_i, y_i) + \frac{h^2}{2} f'(x_i, y_i) \\ &= y_i + hf(x_i, y_i) + \frac{h^2}{2} \left[\frac{\partial f(x_i, y_i)}{\partial x} + \frac{\partial f(x_i, y_i)}{\partial y} f(x_i, y_i) \right] \\ &= y_i + hf(x_i, y_i) + h^2 \left[\frac{1}{2} f_x(x_i, y_i) + \frac{1}{2} f_y(x_i, y_i) f(x_i, y_i) \right] \end{aligned} \quad (3.13)$$

Eqs. (3.12) and (3.13) give two expressions for y_{i+1} . What we want is for (3.12) to yield results of comparable accuracy to (3.13). We achieve this by equating the coefficients of like powers of h in (3.12) and (3.13), thus:

	Runge-Kutta (3.12)	Taylor series (3.13)
h^0	y_i	y_i
h^1	$(a+b)f(x_i, y_i)$	$f(x_i, y_i)$
h^2	$bp f_x(x_i, y_i)$	$\frac{1}{2} f_x(x_i, y_i)$
h^3	$bq f(x_i, y_i) f_y(x_i, y_i)$	$\frac{1}{2} f(x_i, y_i) f_y(x_i, y_i)$

Hence, we obtain:

$$\begin{aligned} a+b &= 1 \\ bp &= \frac{1}{2} \\ bq &= \frac{1}{2} \end{aligned} \quad (3.14)$$

Since we have four unknowns and only three equations, we can impose an additional relation or assume a value for one of the unknowns. Two fine choices are $b = 1/2$ and $b = 1$.

3.6.3 The Improved Euler Method (or Heun's Method)

Taking $b = 1/2$ and solving Eqs. (3.14) yields $a = 1/2$, $p = 1$ and $q = 1$. Substituting these values in the expressions for $\phi(x_i, y_i, h)$ and k_2 yields

$$\begin{aligned}
 y_{i+1} &= y_i + h(ak_1 + bk_2) \\
 &= y_i + h\left(\frac{1}{2}k_1 + \frac{1}{2}k_2\right) \\
 &= y_i + \frac{h}{2}[f(x_i, y_i) + f(x_i + ph, y_i + qhf(x_i, y_i))] \\
 &= y_i + \frac{h}{2}[f(x_i, y_i) + f(x_i + h, y_i + hf(x_i, y_i))]
 \end{aligned} \tag{3.15}$$

Eq. (3.15) is called the *improved Euler method*. We rewrite it in the usual form of a second order Runge-Kutta method:

$$y_{i+1} = y_i + \frac{1}{2}h(k_1 + k_2)$$

where

$$\begin{aligned}
 k_1 &= f(x_i, y_i) \\
 k_2 &= f(x_i + h, y_i + hk_1)
 \end{aligned} \tag{3.15}$$

Application of (3.15) constitutes one integration step of the improved Euler method. The geometrical representation of the method is given below.

3.6.4 The Modified Euler Method (or Midpoint Method)

Taking $b = 1$ and solving Eqs. (3.14) yields $a = 0$, $p = 1/2$ and $q = 1/2$. Substituting these values in the expressions for $\phi(x_i, y_i, h)$ and k_2 yields

$$\begin{aligned}
 y_{i+1} &= y_i + h(ak_1 + bk_2) \\
 &= y_i + hk_2 \\
 &= y_i + hf(x_i + ph, y_i + qhk_1) \\
 &= y_i + hf\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hf(x_i, y_i)\right)
 \end{aligned} \tag{3.16}$$

Eq. (3.16) is called the *modified Euler method*. We rewrite it in the usual form of a second order Runge-Kutta method:

$$y_{i+1} = y_i + hk_2$$

where

$$\begin{aligned}
 k_1 &= f(x_i, y_i) \\
 k_2 &= f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1\right)
 \end{aligned} \tag{3.16}$$

Application of (3.16) constitutes one integration step of the modified Euler method. The geometrical representation of the method is given below.

Problem: See electrical circuit problem for Euler's method

Solution: We write the ODE in canonical form, thus

$$\frac{di}{dt} = \frac{E - Ri}{L} = f(t, i) = \frac{10 - 2i_j}{5} = 2.0 - 0.4i_j$$

Using the improved Euler method, we have:

$$\begin{aligned} k_1 &= f(t_j, i_j) = 2.0 - 0.4i_j \\ k_2 &= f(t_j + h, i_j + hk_1) = 2.0 - 0.4(i_j + hk_1) \\ i_{j+1} &= i_j + \frac{1}{2}h(k_1 + k_2) \end{aligned}$$

Taking $h = 1$ sec and applying the algorithm, we obtain:

$$\begin{aligned} j = 0 : k_1 &= f(t_0, i_0) = 2.0 - 0.4i_0 = 2.0 - 0.4(0) = 2.00000 \\ k_2 &= f(t_0 + h, i_0 + hk_1) = 2.0 - 0.4(i_0 + hk_1) = 2.0 - 0.4[0 + 1(2.00000)] = 1.20000 \\ i_1 &= i_0 + h(k_1 + k_2)/2 = 0 + 1(2.00000 + 1.20000)/2 = 1.60000 \text{ amperes at } t_1 = 1 \text{ sec} \\ j = 1 : k_1 &= f(t_1, i_1) = 2.0 - 0.4i_1 = 2.0 - 0.4(1.60000) = 1.36000 \\ k_2 &= f(t_1 + h, i_1 + hk_1) = 2.0 - 0.4(i_1 + hk_1) = 2.0 - 0.4[1.60000 + 1(1.36000)] = 0.81600 \\ i_2 &= i_1 + h(k_1 + k_2)/2 = 1.60000 + 1(1.36000 + 0.81600)/2 = 2.68800 \text{ amperes at } t_2 = 2 \text{ sec} \\ &\vdots \end{aligned}$$

The following tables summarize the results for $h = 1.0$ sec, 10^{-p} sec, $p = 1, \dots, 5$. The numerical results are then compared with the analytical solution

$$i(t) = \frac{E}{R}(1 - e^{-Rt/L}) = 5.0(1 - e^{-0.4t})$$

$h = 1.0$ sec				$h = 10^{-1}$ sec		$h = 10^{-2}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error	i_j	Error
1	1.648400	1.600000	-0.483998D-01	1.648031	-0.368425D-03	1.648396	-0.359037D-05
2	2.753355	2.688000	-0.653552D-01	2.752861	-0.493961D-03	2.753350	-0.482150D-05
3	3.494029	3.427840	-0.661890D-01	3.493532	-0.496706D-03	3.494024	-0.485873D-05
4	3.990517	3.930931	-0.595862D-01	3.990073	-0.443974D-03	3.990513	-0.435547D-05

$h = 10^{-3}$ sec				$h = 10^{-4}$ sec		$h = 10^{-5}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error	i_j	Error
1	1.648400	1.648400	-0.403471D-07	1.648400	-0.494349D-08	1.648400	-0.458957D-08
2	2.753355	2.753355	-0.621889D-07	2.753355	-0.147254D-07	2.753355	-0.142509D-07
3	3.494029	3.494029	-0.733418D-07	3.494029	-0.256181D-07	3.494029	-0.251410D-07
4	3.990517	3.990517	-0.784796D-07	3.990517	-0.358261D-07	3.990517	-0.353998D-07

Note that decreasing the step size by a factor of 10^{-p} reduces the error by a factor of 10^{-2p} since IEM is of order 2. However, for $h \leq 10^{-4}$, the reduction is only marginal since the additional accuracy gained due to the smaller h is eaten up by the accumulation of round-off error due to the large increase in the number of integration steps.

3.6.5 Third Order Runge-Kutta Methods

Let $\phi(x_i, y_i, h) = ak_1 + bk_2 + ck_3$, where k_1 , k_2 and k_3 are derivative evaluations on the integration interval $[x_i, x_{i+1}]$. Specifically let

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f(x_i + ph, y_i + phk_1) \\ k_3 &= f(x_i + rh, y_i + shk_1 + (r-s)hk_2) \end{aligned}$$

Then the third order Runge-Kutta methods are given by

$$y_{i+1} = y_i + h(ak_1 + bk_2 + ck_3) \quad (3.17)$$

for suitable values of the parameters a, b, c, p, r and s . The governing relations are found by requiring Eq. (3.17) to yield results with accuracy comparable to the TSE of $y(x)$ with terms retained through h^3 . These are:

$$\begin{aligned} a + b + c &= 1 \\ bp + cr &= \frac{1}{2} \\ bp^2 + cr^2 &= \frac{1}{3} \\ cps &= \frac{1}{6} \end{aligned}$$

Taking $p = 1/2$ and $r = 1$ yields $a = c = 1/6$, $b = 2/3$ and $s = 2$ and the following third order Runge-Kutta algorithm (after Kutta):

$$y_{i+1} = y_i + \frac{h}{6}(k_1 + 4k_2 + k_3)$$

where

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1\right) \\ k_3 &= f(x_i + h, y_i + 2hk_1 - hk_2) \end{aligned}$$

3.6.6 Fourth Order Runge-Kutta Methods

Let $\phi(x_i, y_i, h) = ak_1 + bk_2 + ck_3 + dk_4$, where k_1, k_2, k_3 and k_4 are derivative evaluations on the integration interval $[x_i, x_{i+1}]$. Specifically let

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f(x_i + mh, y_i + m h k_1) \\ k_3 &= f(x_i + qh, y_i + (q - r)h k_1 + r h k_2) \\ k_4 &= f(x_i + ph, y_i + (p - s - t)h k_1 + s h k_2 + t h k_3) \end{aligned}$$

Then the fourth order Runge-Kutta methods are given by

$$y_{i+1} = y_i + h(ak_1 + bk_2 + ck_3 + dk_4) \quad (3.18)$$

for suitable values of the parameters $a, b, c, d, m, p, q, r, s$ and t . The governing relations are found by requiring Eq. (3.18) to yield results with accuracy comparable to the TSE of $y(x)$ with terms retained through h^4 . These are:

$$\begin{aligned} a + b + c + d &= 1 & cmr + d(qt + ms) &= \frac{1}{6} \\ bm + cq + dp &= \frac{1}{2} & cmqr + dp(qt + ms) &= \frac{1}{8} \\ bm^2 + cq^2 + dp^2 &= \frac{1}{3} & cm^2r + d(q^2t + m^2s) &= \frac{1}{12} \\ bm^3 + cq^3 + dp^3 &= \frac{1}{4} & dmrt &= \frac{1}{24} \end{aligned}$$

Imposing the additional relations $m = q$ and $b = c$ yields the values $a = d = 1/6$, $b = c = 1/3$, $m = q = r = 1/2$, $p = t = 1$ and $s = 0$ and the *classical* fourth order Runge-Kutta algorithm (after

Runge):

where

$$\begin{aligned}
 y_{i+1} &= y_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\
 k_1 &= f(x_i, y_i) \\
 k_2 &= f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1\right) \\
 k_3 &= f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_2\right) \\
 k_4 &= f(x_i + h, y_i + hk_3)
 \end{aligned} \tag{3.19}$$

Another fourth order Runge-Kutta method, ascribed to Kutta, results from imposing the additional relations $m = \frac{1}{2}q$ and $b = c$:

$$y_{i+1} = y_i + \frac{h}{8}(k_1 + 3k_2 + 3k_3 + k_4)$$

where

$$\begin{aligned}
 k_1 &= f(x_i, y_i) \\
 k_2 &= f\left(x_i + \frac{1}{3}h, y_i + \frac{1}{3}hk_1\right) \\
 k_3 &= f\left(x_i + \frac{2}{3}h, y_i - \frac{1}{3}hk_1 + hk_2\right) \\
 k_4 &= f(x_i + h, y_i + hk_1 - hk_2 + hk_3)
 \end{aligned}$$

Problem: See electrical circuit problem for Euler's method

Solution: We write the ODE in canonical form, thus

$$\frac{di}{dt} = f(t, i) = \frac{E - Ri}{L} = \frac{10 - 2i_j}{5} = 2.0 - 0.4i_j$$

Using the classical 4th order Runge-Kutta method, we have:

$$\begin{aligned}
 k_1 &= f(t_j, i_j) &= 2.0 - 0.4i_j \\
 k_2 &= f\left(t_j + \frac{1}{2}h, i_j + \frac{1}{2}hk_1\right) &= 2.0 - 0.4\left(i_j + \frac{1}{2}hk_1\right) \\
 k_3 &= f\left(t_j + \frac{1}{2}h, i_j + \frac{1}{2}hk_2\right) &= 2.0 - 0.4\left(i_j + \frac{1}{2}hk_2\right) \\
 k_4 &= f\left(t_j + h, i_j + hk_3\right) &= 2.0 - 0.4(i_j + hk_3) \\
 i_{j+1} &= i_j + h(k_1 + 2k_2 + 2k_3 + k_4)/6
 \end{aligned}$$

Taking $h = 1$ sec and applying the algorithm, we obtain:

$$\begin{aligned}
 j = 0 : \quad k_1 &= 2.0 - 0.4i_0 &= 2.0 - 0.4(0) &= 2.00000 \\
 k_2 &= 2.0 - 0.4\left(i_0 + \frac{1}{2}hk_1\right) &= 2.0 - 0.4\left[0 + \frac{1}{2}(1)(2.00000)\right] &= 1.60000 \\
 k_3 &= 2.0 - 0.4\left(i_0 + \frac{1}{2}hk_2\right) &= 2.0 - 0.4\left[0 + \frac{1}{2}(1)(1.60000)\right] &= 1.68000 \\
 k_4 &= 2.0 - 0.4(i_0 + hk_3) &= 2.0 - 0.4[0 + (1)(1.68000)] &= 1.32800 \\
 i_1 &= i_0 + h(k_1 + 2k_2 + 2k_3 + k_4)/6 &= 0 + 1(2.00000 + 2(1.60000) + 2(1.68000) + 1.32800)/6 \\
 &= 1.64800 \text{ amperes at } t = 1 \text{ sec} \\
 j = 1 : \quad k_1 &= 2.0 - 0.4i_1 &= 2.0 - 0.4(1.64800) &= 1.34080 \\
 k_2 &= 2.0 - 0.4\left(i_1 + \frac{1}{2}hk_1\right) &= 2.0 - 0.4\left[1.64800 + \frac{1}{2}(1)(1.34080)\right] &= 1.07264 \\
 k_3 &= 2.0 - 0.4\left(i_1 + \frac{1}{2}hk_2\right) &= 2.0 - 0.4\left[1.64800 + \frac{1}{2}(1)(1.07264)\right] &= 1.126272 \\
 k_4 &= 2.0 - 0.4(i_1 + hk_3) &= 2.0 - 0.4[1.64800 + (1)(1.126272)] &= 0.8902912 \\
 i_1 &= i_1 + h(k_1 + 2k_2 + 2k_3 + k_4)/6 \\
 &= 1.64800 + 1(1.34080 + 2(1.07264) + 2(1.126272) + 0.8902912)/6 \\
 &= 2.752819 \text{ amperes at } t = 2 \text{ sec} \\
 &\vdots
 \end{aligned}$$

The following tables summarize the results for $h = 1$ sec and 10^{-p} sec, $p = 1, 2, 3$ and compare the results with the analytical solution.

$h = 1.0$ sec				$h = 10^{-1}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error
1	1.648400	1.648000	-0.399774D-03	1.648400	-0.341562D-07
2	2.753355	2.752819	-0.535994D-03	2.753355	-0.538891D-07
3	3.494029	3.493490	-0.538974D-03	3.494029	-0.649965D-07
4	3.990517	3.990036	-0.481755D-03	3.990517	-0.710209D-07

$h = 10^{-2}$ sec				$h = 10^{-3}$ sec	
t_j	$i(t_j)$	i_j	Error	i_j	Error
1	1.648400	1.648400	-0.458885D-08	1.648400	-0.458598D-08
2	2.753355	2.753355	-0.142499D-07	2.753355	-0.142461D-07
3	3.494029	3.494029	-0.251401D-07	3.494029	-0.251362D-07
4	3.990517	3.990517	-0.353989D-07	3.990517	-0.353954D-07

3.7 Systems of First Order ODEs

Thus far we have considered the numerical solution to the initial value problem

$$\frac{dy}{dx} = f(x, y) \quad \text{such that} \quad y(x_0) = y_0 \quad (3.5)$$

using the Taylor series method, Euler's method and the Runge-Kutta methods. Let us now turn our attention to the larger, more common, IVP defined by a system of n coupled first order ODEs of the form

$$\begin{aligned} \frac{dy_1(x)}{dx} &= f_1(x, y_1, y_2, y_3, \dots, y_n) \\ \frac{dy_2(x)}{dx} &= f_2(x, y_1, y_2, y_3, \dots, y_n) \\ \frac{dy_3(x)}{dx} &= f_3(x, y_1, y_2, y_3, \dots, y_n) \\ &\vdots \\ \frac{dy_n(x)}{dx} &= f_n(x, y_1, y_2, y_3, \dots, y_n) \end{aligned} \quad (3.2)$$

subject to the initial conditions

$$\begin{aligned} y_1(x_0) &= y_{10} \\ y_2(x_0) &= y_{20} \\ &\vdots \\ y_n(x_0) &= y_{n0} \end{aligned} \quad (3.20)$$

or, concisely

$$\frac{dy_p(x)}{dx} = f_p(x, y_1, y_2, y_3, \dots, y_n) \quad p = 1, 2, \dots, n \quad (3.2)$$

subject to

$$y_p(x_0) = y_{p0} \quad p = 1, 2, \dots, n \quad (3.20)$$

The analytical solution to (3.2) subject to (3.20) consists of n object functions $y_1(x), y_2(x), \dots, y_n(x)$. The

numerical solution is an $m \times n$ table where m is the number of integration steps performed.

x	y_1	y_2	y_3	$\cdots$	y_n	
x_0	y_{10}	y_{20}	y_{30}	$\cdots$	y_{n0}	$\leftarrow$ given ICs
x_1	y_{11}	y_{21}	y_{31}	$\cdots$	y_{n1}	
x_2	y_{12}	y_{22}	y_{32}	$\cdots$	y_{n2}	
$\vdots$						
x_i	y_{1i}	y_{2i}	y_{3i}	$\cdots$	y_{ni}	
x_{i+1}	$y_{1(i+1)}$	$y_{2(i+1)}$	$y_{3(i+1)}$	$\cdots$	$y_{n(i+1)}$	
$\vdots$						
x_m	y_{1m}	y_{2m}	y_{3m}	$\cdots$	y_{nm}	

The solution table is generated row by row starting from the given initial conditions. To generate a row is to perform an integration step. To perform an integration step, we can use any of the methods already discussed for solving a single first order ODE. We simply apply whichever method we choose to each of the n equations in (3.2) *in parallel*.

Consider, for instance, the improved Euler method given by Eq. (3.15) for solving the IVP (3.5). To solve the IVP (3.2 & 3.20), we generalize the method as follows:

$$\begin{aligned}
 y_{p(i+1)} &= y_{pi} + \frac{1}{2}h(k_{p1} + k_{p2}) & p = 1, 2, \dots, n \\
 \text{where} \quad k_{p1} &= f_p(x_i, y_{1i}, y_{2i}, \dots, y_{ni}) & p = 1, 2, \dots, n \\
 k_{p2} &= f_p(x_i + h, y_{1i} + hk_{11}, y_{2i} + hk_{21}, \dots, y_{ni} + hk_{n1}) & p = 1, 2, \dots, n
 \end{aligned} \tag{3.21}$$

To perform an integration step over the interval $[x_i, x_{i+1}]$ using (3.21), we proceed as follows:

1. Evaluate each of the functions $f_1, f_2, \dots, f_n$ at the current point $(x_i, y_{1i}, y_{2i}, \dots, y_{ni})$; this gives us the derivative values (or slopes) $k_{11}, k_{21}, \dots, k_{n1}$ at x_i . With these values known, we locate the sample point $Q(x_i + h, y_{1i} + hk_{11}, y_{2i} + hk_{21}, \dots, y_{ni} + hk_{n1})$.
2. Evaluate each of the functions $f_1, f_2, \dots, f_n$ at the sample point Q ; this gives us the derivative values $k_{12}, k_{22}, \dots, k_{n2}$ at x_{i+1} .
3. Compute the second order Runge-Kutta estimates to the true solution values $y_1(x_{i+1}), y_2(x_{i+1}), \dots, y_n(x_{i+1})$, thus

$$\begin{aligned}
 y_{1(i+1)} &= y_{1i} + \frac{1}{2}h(k_{11} + k_{12}) \\
 y_{2(i+1)} &= y_{2i} + \frac{1}{2}h(k_{21} + k_{22}) \\
 &\vdots \\
 y_{n(i+1)} &= y_{ni} + \frac{1}{2}h(k_{n1} + k_{n2})
 \end{aligned}$$

By the same token, we can generalize the classical fourth order Runge-Kutta algorithm given by Eq. (3.19) as follows:

$$\begin{aligned}
 y_{p(i+1)} &= y_{pi} + \frac{h}{6}(k_{p1} + 2k_{p2} + 2k_{p3} + k_{p4}) & p = 1, 2, \dots, n \\
 \text{where} \quad k_{p1} &= f_p(x_i, y_{1i}, y_{2i}, \dots, y_{ni}) & p = 1, 2, \dots, n \\
 k_{p2} &= f_p\left(x_i + \frac{1}{2}h, y_{1i} + \frac{1}{2}hk_{11}, y_{2i} + \frac{1}{2}hk_{21}, \dots, y_{ni} + \frac{1}{2}hk_{n1}\right) & p = 1, 2, \dots, n \\
 k_{p3} &= f_p\left(x_i + \frac{1}{2}h, y_{1i} + \frac{1}{2}hk_{12}, y_{2i} + \frac{1}{2}hk_{22}, \dots, y_{ni} + \frac{1}{2}hk_{n2}\right) & p = 1, 2, \dots, n \\
 k_{p4} &= f_p\left(x_i + h, y_{1i} + hk_{13}, y_{2i} + hk_{23}, \dots, y_{ni} + hk_{n3}\right) & p = 1, 2, \dots, n
 \end{aligned} \tag{3.22}$$

Example 1: In a hemispherical bowl of radius R , a body of mass m starts from rest on its rim and slides down under the influence of gravity. The differential equation of motion of the body is

$$\frac{d^2\theta}{dt^2} + \frac{g}{R}(f \sin \theta - \cos \theta) = 0$$

where f is the friction vector. Given f and R , the numerical solution to this problem is a table of computed θ and $d\theta/dt$ values at specified values of t . To generate this table let $\omega \equiv d\theta/dt$; then the problem reduces to the IVP

$$\begin{aligned} \frac{d\theta}{dt} &= \omega & &= f_1(t, \theta, \omega) \\ \frac{d\omega}{dt} &= \frac{g}{R}(\cos \theta - f \sin \theta) & &= f_2(t, \theta, \omega) \end{aligned}$$

subject to the initial conditions

$$\begin{aligned} \theta(0) &= 0 & & \text{(since the body starts at the rim)} \\ \omega(0) &= 0 & & \text{(since the body starts from rest)} \end{aligned}$$

Now assume that we want to solve this IVP 'by hand' using the improved Euler method [see Eqs. (3.21)]. Using the notation $\alpha_j \equiv k_{1j}$ and $\beta_j \equiv k_{2j}$, $j = 1, 2$, the *complete* algorithm is:

$$\begin{aligned} \theta_{i+1} &= \theta_i + \frac{1}{2}h(\alpha_1 + \alpha_2) \\ \omega_{i+1} &= \omega_i + \frac{1}{2}h(\beta_1 + \beta_2) \end{aligned}$$

where

$$\begin{aligned} \alpha_1 &= f_1(t_i, \theta_i, \omega_i) & &= \omega_i \\ \beta_1 &= f_2(t_i, \theta_i, \omega_i) & &= \frac{g}{R}(\cos \theta_i - f \sin \theta_i) \\ \alpha_2 &= f_1(t_i + h, \theta_i + h\alpha_1, \omega_i + h\beta_1) & &= \omega_i + h\beta_1 \\ \beta_2 &= f_2(t_i + h, \theta_i + h\alpha_1, \omega_i + h\beta_1) & &= \frac{g}{R}[\cos(\theta_i + h\alpha_1) - f \sin(\theta_i + h\alpha_1)] \end{aligned}$$

Given that $f = 0.006$ and $R = 0.75$ ft and using $h = 0.10$, the computations for the first two time steps are:

1st time step: $i = 0$

$$\begin{aligned} \alpha_1 &= \omega_i = \omega_0 = 0 \\ \beta_1 &= \frac{g}{R}(\cos \theta_i - f \sin \theta_i) = \frac{g}{R}(\cos \theta_0 - f \sin \theta_0) = \frac{32.17}{0.75}(\cos 0 - 0.006 \sin 0) = 42.89333 \\ \alpha_2 &= \omega_i + h\beta_1 = \omega_0 + h\beta_1 = 0 + (0.10)(42.89333) = 4.28933 \\ \beta_2 &= \frac{g}{R}[\cos(\theta_i + h\alpha_1) - f \sin(\theta_i + h\alpha_1)] = \frac{g}{R}[\cos(\theta_0 + h\alpha_1) - f \sin(\theta_0 + h\alpha_1)] \\ &= \frac{32.17}{0.75}[\cos(0 + (0.1)(0)) - 0.006 \sin(0 + (0.1)(0))] = 42.89333 \\ \theta_1 &= \theta_0 + \frac{1}{2}h(\alpha_1 + \alpha_2) = 0 + \frac{1}{2}(0.10)(0 + 4.28933) = 0.21447 \text{ rad at } t = 0.10 \text{ sec} \\ \omega_1 &= \omega_0 + \frac{1}{2}h(\beta_1 + \beta_2) = 0 + \frac{1}{2}(0.10)(42.89333 + 42.89333) = 4.28933 \text{ rad/sec at } t = 0.10 \text{ sec} \end{aligned}$$

2nd time step: $i = 1$

$$\begin{aligned} \alpha_1 &= \omega_i = \omega_1 = 4.28933 \\ \beta_1 &= \frac{g}{R}(\cos \theta_i - f \sin \theta_i) = \frac{g}{R}(\cos \theta_1 - f \sin \theta_1) = \frac{32.17}{0.75}(\cos 0.21447 - 0.006 \sin 0.21447) = 41.85587 \\ \alpha_2 &= \omega_i + h\beta_1 = \omega_1 + h\beta_1 = 4.28933 + (0.10)(41.85587) = 8.47492 \\ \beta_2 &= \frac{g}{R}[\cos(\theta_i + h\alpha_1) - f \sin(\theta_i + h\alpha_1)] = \frac{g}{R}[\cos(\theta_1 + h\alpha_1) - f \sin(\theta_1 + h\alpha_1)] \end{aligned}$$

$$\begin{aligned}
&= \frac{32.17}{0.75} [\cos(0.21447 + (0.10)(4.28933)) - 0.006 \sin(0.21447 + (0.10)(4.28933))] = 34.16287 \\
\theta_2 &= \theta_1 + \frac{1}{2}h(\alpha_1 + \alpha_2) = 0 + \frac{1}{2}(0.10)(4.28933 + 8.47492) = 0.85268 \text{ rad at } t = 0.20 \text{ sec} \\
\omega_2 &= \omega_1 + \frac{1}{2}h(\beta_1 + \beta_2) = 0 + \frac{1}{2}(0.10)(41.85587 + 34.16287) = 8.09027 \text{ rad/sec at } t = 0.20 \text{ sec}
\end{aligned}$$

$t, \text{ sec}$	$\theta, \text{ rad}$	$\omega, \text{ rad/sec}$	
0.00	0.00000	0.00000	Initial conditions
0.10	0.21447	4.28933	
0.20	0.85268	8.09027	

These computations are clearly best carried out by a computer. Convert algorithm (3.21) into a running program and complete the table for the solution interval $[0, 300]$.

Now assume that we want to solve the given problem 'by hand' using the classical 4th order Runge-Kutta method [see Eqs. (3.22)]. Using the notation $\alpha_j \equiv k_{1j}$ and $\beta_j \equiv k_{2j}$, $j = 1, 2, 3, 4$, the *complete* algorithm is:

$$\begin{aligned}
\theta_{i+1} &= \theta_i + \frac{1}{6}h(\alpha_1 + 2\alpha_2 + 2\alpha_3 + \alpha_4) \\
\omega_{i+1} &= \omega_i + \frac{1}{6}h(\beta_1 + 2\beta_2 + 2\beta_3 + \beta_4)
\end{aligned}$$

where

$$\begin{aligned}
\alpha_1 &= f_1(t_i, \theta_i, \omega_i) &= \omega_i \\
\beta_1 &= f_2(t_i, \theta_i, \omega_i) &= \frac{g}{R}(\cos \theta_i - f \sin \theta_i) \\
\alpha_2 &= f_1\left(t_i + h, \theta_i + \frac{1}{2}h\alpha_1, \omega_i + \frac{1}{2}h\beta_1\right) &= \omega_i + \frac{1}{2}h\beta_1 \\
\beta_2 &= f_2\left(t_i + h, \theta_i + \frac{1}{2}h\alpha_1, \omega_i + \frac{1}{2}h\beta_1\right) &= \frac{g}{R}[\cos(\theta_i + \frac{1}{2}h\alpha_1) - f \sin(\theta_i + \frac{1}{2}h\alpha_1)] \\
\alpha_3 &= f_1\left(t_i + h, \theta_i + \frac{1}{2}h\alpha_2, \omega_i + \frac{1}{2}h\beta_2\right) &= \omega_i + \frac{1}{2}h\beta_2 \\
\beta_3 &= f_2\left(t_i + h, \theta_i + \frac{1}{2}h\alpha_2, \omega_i + \frac{1}{2}h\beta_2\right) &= \frac{g}{R}[\cos(\theta_i + \frac{1}{2}h\alpha_2) - f \sin(\theta_i + \frac{1}{2}h\alpha_2)] \\
\alpha_4 &= f_1(t_i + h, \theta_i + h\alpha_3, \omega_i + h\beta_3) &= \omega_i + h\beta_3 \\
\beta_4 &= f_2(t_i + h, \theta_i + h\alpha_3, \omega_i + h\beta_3) &= \frac{g}{R}[\cos(\theta_i + h\alpha_3) - f \sin(\theta_i + h\alpha_3)]
\end{aligned}$$

Given that $f = 0.006$ and $R = 0.75$ ft and using $h = 0.10$, the computations for the first two time steps are:

1st time step: $i = 0$

$$\begin{aligned}
\alpha_1 &= \omega_i = \omega_0 = 0 \\
\beta_1 &= \frac{g}{R}(\cos \theta_i - f \sin \theta_i) = \frac{g}{R}(\cos \theta_0 - f \sin \theta_0) = \frac{32.17}{0.75}(\cos 0 - 0.006 \sin 0) = 42.89333 \\
\alpha_2 &= \omega_i + \frac{1}{2}h\beta_1 = \omega_0 + \frac{1}{2}h\beta_1 = 0 + \frac{1}{2}(0.10)(42.89333) = 2.14467 \\
\beta_2 &= \frac{g}{R} \left[\cos \left(\theta_i + \frac{1}{2}h\alpha_1 \right) - f \sin \left(\theta_i + \frac{1}{2}h\alpha_1 \right) \right] = \frac{g}{R} \left[\cos \left(\theta_0 + \frac{1}{2}h\alpha_1 \right) - f \sin \left(\theta_0 + \frac{1}{2}h\alpha_1 \right) \right] \\
&= \frac{32.17}{0.75} \left[\cos \left(0 + \frac{1}{2}(0.1)(0) \right) - 0.006 \sin \left(0 + \frac{1}{2}(0.1)(0) \right) \right] = 42.89333 \\
\alpha_3 &= \omega_i + \frac{1}{2}h\beta_2 = \omega_0 + \frac{1}{2}h\beta_2 = 0 + \frac{1}{2}(0.10)(42.89333) = 2.14467 \\
\beta_3 &= \frac{g}{R} \left[\cos \left(\theta_i + \frac{1}{2}h\alpha_2 \right) - f \sin \left(\theta_i + \frac{1}{2}h\alpha_2 \right) \right] = \frac{g}{R} \left[\cos \left(\theta_0 + \frac{1}{2}h\alpha_2 \right) - f \sin \left(\theta_0 + \frac{1}{2}h\alpha_2 \right) \right] \\
&= \frac{32.17}{0.75} \left[\cos \left(0 + \frac{1}{2}(0.1)(2.14467) \right) - 0.006 \sin \left(0 + \frac{1}{2}(0.1)(2.14467) \right) \right] = 42.61941 \\
\alpha_4 &= \omega_i + h\beta_3 = \omega_0 + h\beta_3 = 0 + (0.10)(42.61941) = 4.26194
\end{aligned}$$

$$\begin{aligned}
\beta_4 &= \frac{g}{R} [\cos(\theta_i + h\alpha_3) - f \sin(\theta_i + h\alpha_3)] = \frac{g}{R} [\cos(\theta_0 + h\alpha_3) - f \sin(\theta_0 + h\alpha_3)] \\
&= \frac{32.17}{0.75} [\cos(0 + (0.1)(2.14467)) - 0.006 \sin(0 + (0.1)(2.14467))] = 41.85587 \\
\theta_1 &= \theta_0 + \frac{1}{6} h(\alpha_1 + 2\alpha_2 + 2\alpha_3 + \alpha_4) = 0 + \frac{1}{6} (0.10)[0 + 2(2.14467) + 2(2.14467) + 4.26194] \\
&= 0.21401 \text{ rad at } t = 0.10 \text{ sec} \\
\omega_1 &= \omega_0 + \frac{1}{6} h(\beta_1 + 2\beta_2 + 2\beta_3 + \beta_4) = 0 + \frac{1}{6} (0.10)[42.89333 + 2(42.89333) + 2(42.61941) + 41.85587] \\
&= 4.26291 \text{ rad/sec at } t = 0.10 \text{ sec}
\end{aligned}$$

2nd time step: $i = 1$

$$\begin{aligned}
\alpha_1 &= \omega_1 = 4.26291 \\
\beta_1 &= \frac{g}{R} (\cos \theta_1 - f \sin \theta_1) = \frac{32.17}{0.75} (\cos 0.21401 - 0.006 \sin 0.21401) = 41.86015 \\
\alpha_2 &= \omega_1 + \frac{1}{2} h\beta_1 = 4.26291 + \frac{1}{2} (0.10)(41.86015) = 6.35592 \\
\beta_2 &= \frac{g}{R} \left[\cos \left(\theta_1 + \frac{1}{2} h\alpha_1 \right) - f \sin \left(\theta_1 + \frac{1}{2} h\alpha_1 \right) \right] \\
&= \frac{32.17}{0.75} \left[\cos \left(0.21401 + \frac{1}{2} (0.1)(4.26291) \right) - 0.006 \sin \left(0.21401 + \frac{1}{2} (0.1)(4.26291) \right) \right] = 38.93265 \\
\alpha_3 &= \omega_1 + \frac{1}{2} h\beta_2 = 4.26291 + \frac{1}{2} (0.10)(38.93265) = 6.20954 \\
\beta_3 &= \frac{g}{R} \left[\cos \left(\theta_1 + \frac{1}{2} h\alpha_2 \right) - f \sin \left(\theta_1 + \frac{1}{2} h\alpha_2 \right) \right] \\
&= \frac{32.17}{0.75} \left[\cos \left(0.21401 + \frac{1}{2} (0.1)(6.20954) \right) - 0.006 \sin \left(0.21401 + \frac{1}{2} (0.1)(6.20954) \right) \right] = 36.83894 \\
\alpha_4 &= \omega_1 + h\beta_3 = 4.26291 + (0.10)(36.83894) = 7.94681 \\
\beta_4 &= \frac{g}{R} [\cos(\theta_1 + h\alpha_3) - f \sin(\theta_1 + h\alpha_3)] \\
&= \frac{32.17}{0.75} [\cos(0.21401 + (0.1)(6.20954)) - 0.006 \sin(0.21401 + (0.1)(6.20954))] = 28.59940 \\
\theta_2 &= \theta_1 + \frac{1}{6} h(\alpha_1 + 2\alpha_2 + 2\alpha_3 + \alpha_4) \\
&= 0.21401 + \frac{1}{6} (0.10)[4.26291 + 2(6.35592) + 2(6.20954) + 7.94681] = 0.83635 \text{ rad at } t = 0.20 \text{ sec} \\
\omega_2 &= \omega_1 + \frac{1}{6} h(\beta_1 + 2\beta_2 + 2\beta_3 + \beta_4) \\
&= 4.26291 + \frac{1}{6} (0.10)[41.86015 + 2(38.93265) + 2(36.83894) + 28.59940] \\
&= 7.96296 \text{ rad/sec at } t = 0.20 \text{ sec}
\end{aligned}$$

3.7.1 EASY Implementation of the Classical Fourth Order Runge-Kutta Method

procedure RK4(m, n, h, X, Y)

//Solves the IVP defined by a system of n first order ODEs using the classical 4th order Runge-Kutta algorithm. m is the number of integration steps to be performed; h is the step size. Upon entry into procedure $X(0), Y(1, 0), Y(2, 0), \dots, Y(n, 0)$ contain the initial values; upon exit, the entire solution table is stored in the arrays X and Y . //

array $X(0:m), Y(1:n, 0:m), T(0:n), U(1:n), \sigma(1:n), \tau(1:n)$

$\sigma(1) \leftarrow \sigma(4) \leftarrow h/6$

$\sigma(2) \leftarrow \sigma(3) \leftarrow h/3$

$\tau(1) \leftarrow \tau(2) \leftarrow h/2$

```

 $\tau(3) \leftarrow \tau(4) \leftarrow h$ 
//Perform  $m$  integration steps//
for  $i \leftarrow 0$  to  $m - 1$  do
   $T(0) \leftarrow X(i)$ 
  for  $p \leftarrow 1$  to  $n$  do
     $T(p) \leftarrow Y(p, i)$ 
     $Y(p, i + 1) \leftarrow Y(p, i)$ 
  end for
  for  $j \leftarrow 1$  to 4 do
    for  $p \leftarrow 1$  to  $n$  do
       $k \leftarrow F(p, T)$ 
       $U(p) \leftarrow Y(p, i) + \tau(j) * k$ 
       $Y(p, i + 1) \leftarrow Y(p, i + 1) + \sigma(j) * k$ 
    end for
    if  $j \neq 4$  then [for  $p \leftarrow 1$  to  $n$  do;  $T(p) \leftarrow U(p)$ ; end for;  $T(0) \leftarrow X(i) + \tau(j)$ ]
  end for
   $X(i + 1) \leftarrow X(i) + h$ 
end for
end procedure

```

As implemented, RK4 is called only once to generate the desired solution. However, enough space must be allocated to store the *entire* solution table; if m is large, this may not be feasible. Since the Runge-Kutta methods are one step methods, only two rows of the solution table are actually needed at any time during the computations—the current row and the next row. `procedure RK4` can be readily modified such that it performs only one integration step, to be repeatedly called by a driver routine which determines which rows of the solution table will be retained in memory (say every 1000th row, possibly for subsequent output), until the desired solution is obtained. This is left as an exercise for you to do.

Now assume that we want to find the solution to the sliding-body problem given above over the interval $[0, 300]$ using a step size of 0.10 sec using `procedure RK4`. The following procedures, with `RK4`, will do the task.

```

procedure MAIN()
  array  $t(0:3000), y(1:2, 0:3000)$ 
  //initial conditions//
   $t(0) \leftarrow 0$ 
   $y(1, 0) \leftarrow 0$ 
   $y(2, 0) \leftarrow 0$ 
  call RK4(3000, 2, 0.10,  $t, y$ )
  for  $i \leftarrow 0$  to 3000 do
    output  $t(i), y(1, i), y(2, i)$ 
  end for
end procedure

procedure  $F(p, v)$ 
  array  $v(0:2)$ 
  case
    :  $p = 1$  : return  $v(2)$ 
    :  $p = 2$  : return  $(32.17/0.75) * (\cos(v(1)) - 0.006 * \sin(v(1)))$ 
  end case
end procedure

```

$//v(0) \equiv t \quad v(1) \equiv \theta \quad v(2) \equiv \omega//$

Shown below is a sample output for the run.

t , sec	θ , rad	ω , rad/sec	Initial conditions
0.0	0.0	0.0	
0.10	0.214010	4.262911	
0.20	0.836354	7.962957	
0.30	1.721730	9.170726	
0.40	2.546065	6.846114	
0.50	3.034453	2.805518	
0.60	3.101117	-1.475448	
0.70	2.741215	-5.667903	
0.80	2.001749	-8.767566	
0.90	1.098186	-8.696981	
1.00	0.370574	-5.519141	
1.10	0.026330	-1.316170	
1.20	0.108912	2.963457	
1.30	0.611969	6.970024	
1.40	1.443192	9.167238	
1.50	2.320468	7.817873	
1.60	2.925261	4.071984	
1.70	3.120011	-0.192633	
1.80	2.886934	-4.446889	
1.90	2.249252	-8.066883	
2.00	1.362396	-9.089021	

We see from the table that between 0.20 sec and 0.30 sec from release, the body is at the bottom of the bowl (at its fastest) on its way to the opposite end of the rim, which it nearly reaches at $0.50 \text{ sec} < t < 0.60 \text{ sec}$. At $0.80 \text{ sec} < t < 0.90 \text{ sec}$, it is again at the bottom of the bowl on its return trip to its starting point, approximately reaching this between 1.10 sec and 1.20 sec from the start of motion. And so on.

Plot of angular displacement (rad) vs time elapsed (sec)
 Plot of angular velocity (rad/sec) vs time elapsed (sec)
 Plot of angular displacement (rad) vs time elapsed (sec)
 Plot of angular velocity (rad/sec) vs time elapsed (sec)

Example 2: The figure shows a satellite in orbit around the earth and the moon. The *rotating* coordinate system x - y lies in the plane of the three bodies with the origin at the center of mass of the two much heavier bodies. If the distance between the centers of mass of the earth and the moon is taken to be unity, and the unit of time is chosen such that the angular velocity ω of the rotating x - y reference frame is unity, then the equations of motion of the satellite are (Forsythe, pp. 153–154)

$$\begin{aligned}\frac{d^2x}{dt^2} &= 2\frac{dy}{dt} + x - \frac{\beta(x+\alpha)}{\rho_1} - \frac{\alpha(x-\beta)}{\rho_2} \\ \frac{d^2y}{dt^2} &= -2\frac{dx}{dt} + y - \frac{\beta y}{\rho_1} - \frac{\alpha y}{\rho_2}\end{aligned}$$

where

$$\begin{aligned}\alpha &= \frac{m_{\text{moon}}}{m_{\text{earth}} + m_{\text{moon}}} = \frac{1}{82.45} & \beta &= 1 - \alpha \\ \rho_1 &= \sqrt{((x+\alpha)^2 + y^2)^3} & \rho_2 &= \sqrt{((x-\beta)^2 + y^2)^3}\end{aligned}$$

This system of 2nd order ODEs has no closed-form solution; however, it is known that for certain initial conditions, the solution is *periodic*. We consider one such case. To this end, let $u \equiv dx/dt$ and $w \equiv dy/dt$;

we want to solve the IVP

$$\begin{aligned}
 \frac{dx}{dt} &= u & &= f_1(t, x, y, u, w) \\
 \frac{dy}{dt} &= w & &= f_2(t, x, y, u, w) \\
 \frac{du}{dt} &= 2w + x - \frac{\beta(x + \alpha)}{\rho_1} - \frac{\alpha(x - \beta)}{\rho_2} & &= f_3(t, x, y, u, w) \\
 \frac{dw}{dt} &= -2u + y - \frac{\beta y}{\rho_1} - \frac{\alpha y}{\rho_2} & &= f_4(t, x, y, u, w)
 \end{aligned}$$

subject to the initial conditions

$$\begin{aligned}
 x(0) &= 1.2 \\
 y(0) &= 0 \\
 u(0) &= 0 \\
 w(0) &= -1.04935751
 \end{aligned}$$

The solution to this IVP is periodic with a period equal to 6.19216933. The following procedure calls on RK4 to generate the solution over two periods; this will provide a check on the numerical results.

```

procedure MAIN()
  array t(0:200000), y(1:4,0:200000)
  //initial conditions//
  t(0) ← 0
  y(1,0) ← 1.2
  y(2,0) ← 0
  y(3,0) ← 0
  y(4,0) ← -1.04935751
  h ← 6.19216933/100000
  call RK4(200000, 4, h, t, y)
  for i ← 0 to 200000 by 10000 do
    output t(i), y(1,i), y(2,i), y(3,i), y(4,i)
  end for
end procedure

procedure F(p, v)
  array v(0:4)
  //v(0) ≡ t    v(1) ≡ x    v(2) ≡ y    v(3) ≡ u    v(4) ≡ w//
  case
  : p = 1 : return v(3)
  : p = 2 : return v(4)
  : p = 3 : return 2 * v(4) + v(1) - β * (v(1) + α) / ρ1 - α * (v(1) - β) / ρ2
  : p = 4 : return -2 * v(3) + v(2) - β * v(2) / ρ1 - α * v(2) / ρ2
  end case
end procedure

```

These procedures were transcribed into FORTRAN 90 and run, in double precision, for three values of the step size h . Below is a summary of the results.

(a) $h = 6.19216933/1000$

t	x	y	dx/dt	dy/dt
0.000000	1.200000	0.000000	0.000000	-1.049358
0.619217	0.903563	-0.517369	-0.844409	-0.461205
1.238434	0.316989	-0.392490	-0.929426	1.093599
1.857651	-0.258967	-0.495740	-0.257209	-0.302089
2.476868	-0.072060	-0.200341	1.580626	1.810119
3.096085	-0.109898	0.093384	-2.679940	0.249911
3.715302	-0.078787	0.124894	-1.875841	0.268314
4.334518	-0.056526	-0.015713	3.367531	-3.707411
4.953735	-5.961289	-2.693388	-25.256187	-4.326878
5.572952	-21.490645	4.187340	-20.153281	26.207213
6.192169	-25.648667	27.008959	9.925591	43.616213
6.811386	-7.367486	52.080352	48.597164	31.913889
7.430603	31.293087	60.316106	71.726226	-9.282656
8.049820	74.154208	37.952987	60.019331	-62.852840
8.669037	97.602585	-14.364313	10.164257	-101.206749
9.288254	82.217196	-78.978405	-61.095863	-99.388473
9.907471	23.959341	-127.118832	-122.522661	-48.321426
10.526688	-60.706505	-131.359403	-141.756271	38.200074
11.145905	-139.007245	-79.348698	-100.877869	126.713842
11.765121	-174.526218	17.601676	-7.065283	177.010770
12.384338	-143.451877	125.746174	107.101092	159.791764

The numerical results are obviously grossly in error.

(b) $h = 6.19216933/10000$

t	x	y	dx/dt	dy/dt
0.000000	1.200000	0.000000	0.000000	-1.049358
0.619217	0.903563	-0.517369	-0.844409	-0.461205
1.238434	0.316989	-0.392490	-0.929426	1.093599
1.857651	-0.430883	-0.587674	-0.897343	-0.629577
2.476868	-0.999877	-0.556160	-0.786230	0.602006
3.096085	-1.262432	-0.000014	0.000013	1.049534
3.715302	-0.999862	0.556119	0.786248	0.601962
4.334518	-0.430861	0.587582	0.897353	-0.629726
4.953735	0.317027	0.392614	0.929316	1.093313
5.572952	0.903568	0.517418	0.844356	-0.461252
6.192169	1.199970	0.000047	-0.000054	-1.049325
6.811386	0.903493	-0.517262	-0.844457	-0.461032
7.430603	0.316887	-0.392181	-0.929626	1.094310
8.049820	-0.430960	-0.587874	-0.897319	-0.629184
8.669037	-0.999921	-0.556223	-0.786143	0.602119
9.288254	-1.262413	-0.000054	0.000118	1.049498
9.907471	-0.999792	0.556013	0.786298	0.601783
10.526688	-0.430784	0.587296	0.897346	-0.630214
11.145905	0.317177	0.392976	0.929052	1.092402
11.765121	0.903647	0.517526	0.844238	-0.461442
12.384338	1.199969	0.000111	-0.000169	-1.049300

The periodic nature of the solution is already evident. Note that 20,000 integration steps were performed to generate this table.

(c) $h = 6.19216933/25000$

t	x	y	dx/dt	dy/dt
0.000000	1.200000	0.000000	0.000000	-1.049358
0.619217	0.903563	-0.517369	-0.844409	-0.461205
1.238434	0.316989	-0.392490	-0.929426	1.093599
1.857651	-0.430888	-0.587673	-0.897355	-0.629576
2.476868	-0.999890	-0.556157	-0.786244	0.602015
3.096085	-1.262454	0.000000	0.000000	1.049559
3.715302	-0.999890	0.556156	0.786244	0.602015
4.334518	-0.430888	0.587673	0.897355	-0.629576
4.953735	0.316989	0.392491	0.929426	1.093598
5.572952	0.903563	0.517370	0.844409	-0.461205
6.192169	1.200000	0.000001	0.000000	-1.049357
6.811386	0.903563	-0.517368	-0.844409	-0.461204
7.430603	0.316988	-0.392488	-0.929428	1.093604
8.049820	-0.430888	-0.587675	-0.897355	-0.629573
8.669037	-0.999890	-0.556157	-0.786243	0.602016
9.288254	-1.262454	-0.000001	0.000000	1.049559
9.907471	-0.999890	0.556155	0.786244	0.602014
10.526688	-0.430887	0.587670	0.897356	-0.629580
11.145905	0.316990	0.392494	0.929423	1.093590
11.765121	0.903563	0.517371	0.844408	-0.461206
12.384338	1.200000	0.000002	-0.000001	-1.049357

The solution is clearly periodic. Note that 50,000 integration steps were performed to generate this table.

3.8 The Fourth Order Runge-Kutta-Fehlberg Formulas

E. Fehlberg developed, in 1970, Runge-Kutta formulas which require six derivative evaluations $k_1, k_2, \dots, k_6$. Four of these k_j s are combined to yield a 4th order method and five of them are combined to produce a 5th order method. The 4th and 5th order estimates of $y(x_{i+1})$ given by these methods are used to obtain an estimate of the local truncation error; this allows for the step size to be controlled to keep errors within a prescribed tolerance.

The 4th order Runge-Kutta-Fehlberg formulas are:

$$\begin{aligned}
 y_{i+1} &= y_i + h \left(\frac{25}{216}k_1 + \frac{1408}{2565}k_3 + \frac{2197}{4104}k_4 - \frac{1}{5}k_5 \right) \\
 \ddot{y}_{i+1} &= y_1 + h \left(\frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6 \right)
 \end{aligned}$$

where

$$\begin{aligned}
 k_1 &= f(x_i, y_i) \\
 k_2 &= f\left(x_i + \frac{1}{4}h, y_i + \frac{1}{4}hk_1\right) \\
 k_3 &= f\left(x_i + \frac{3}{8}h, y_i + \frac{3}{32}hk_1 + \frac{9}{32}hk_2\right) \\
 k_4 &= f\left(x_i + \frac{12}{13}h, y_i + \frac{1932}{2197}hk_1 - \frac{7200}{2197}hk_2 + \frac{7296}{2197}hk_3\right) \\
 k_5 &= f\left(x_i + h, y_i + \frac{439}{216}hk_1 - 8hk_2 + \frac{3680}{513}hk_3 - \frac{845}{4104}hk_4\right) \\
 k_6 &= f\left(x_i + \frac{1}{2}h, y_i - \frac{8}{27}hk_1 + 2hk_2 - \frac{3544}{2565}hk_3 + \frac{1859}{4104}hk_4 - \frac{11}{40}hk_5\right)
 \end{aligned} \tag{3.23}$$

3.8.1 Adaptive Stepsize Control Using Two Runge-Kutta Methods of Different Orders

We will now consider how to utilize the estimate of the local truncation error, obtained from two estimates of $y(x_{i+1})$ given by two Runge-Kutta formulas of different orders, e.g., Eqs. (3.23), to control the stepsize h such that the global error is kept within a prescribed tolerance. To this end, let:

- h_i be the current stepsize
- y_{i+1} be the estimate of $y(x_{i+1})$ given by a Runge-Kutta formula of order p
- $\ddot{y}_{i+1}$ be the estimate of $y(x_{i+1})$ given by a Runge-Kutta formula of order $p + 1$
- $y^*(x_{i+1})$ be the exact solution to the IVP obtained by taking as initial data the approximation at the point (x_i, y_i) , i.e., the IVP $\frac{dy^*}{dx} = f(x, y^*)$, $x_i \leq x \leq x_n$ subject to the initial condition $y^*(x_i) = y_i$

Then, for sufficiently small h , we have

$$y^*(x_{i+1}) = y_{i+1} + O(h_i^{p+1}) \quad (3.24)$$

$$y^*(x_{i+1}) = \ddot{y}_{i+1} + O(h_i^{p+2}) \quad (3.25)$$

since the local truncation error for a Runge-Kutta formula of order p is $O(h^{p+1})$ and that for a formula of order $p + 1$ is $O(h^{p+2})$. Subtracting (3.25) from (3.24), we get

$$\ddot{y}_{i+1} - y_{i+1} = O(h_i^{p+1}) - O(h_i^{p+2}) = O(h_i^{p+1})$$

or, equivalently

$$|\ddot{y}_{i+1} - y_{i+1}| \leq k h_i^{p+1} \quad (3.26)$$

where k is some positive constant. From (3.26), we could take as an estimate of k the quantity

$$k = \frac{|\ddot{y}_{i+1} - y_{i+1}|}{h_i^{p+1}} \quad (3.27)$$

Note that the quantity $\ddot{y}_{i+1} - y_{i+1}$ in (3.26) is an estimate of the local truncation error, say e_{i+1} , of the p th order estimate y_{i+1} . If the stepsize were αh_i , then the corresponding local truncation error, say $\acute{e}_{i+1}$, would be $\acute{e}_{i+1} \leq \acute{k}(\alpha h_i)^{p+1}$ for some positive constant $\acute{k}$. Assuming that $\acute{k} = k$ from (3.27), we get

$$\acute{e}_{i+1} = \frac{|\ddot{y}_{i+1} - y_{i+1}|}{h_i^{p+1}} (\alpha h_i)^{p+1} = |\ddot{y}_{i+1} - y_{i+1}| \alpha^{p+1}$$

It can be shown (see Burden, *et al*, pp. 287–288) that the global truncation error will be bounded by a constant times a tolerance, say ε , provided that the local truncation error is bounded by ε times the stepsize at every integration step. Hence, we must have

$$|\ddot{y}_{i+1} - y_{i+1}| \alpha^{p+1} \leq \varepsilon \alpha h_i$$

which implies that

$$\alpha \leq \sqrt[p]{\frac{\varepsilon h_i}{|\ddot{y}_{i+1} - y_{i+1}|}} \quad (3.28)$$

The value of α is calculated at every integration step, and is used for two purposes. Assume that y_{i+1} and $\ddot{y}_{i+1}$ are two estimates of $y(x_{i+1})$ given by a p th order and a $(p + 1)$ th order Runge-Kutta formula, respectfully, using a stepsize h_i . Then:

- (a) if $|\ddot{y}_{i+1} - y_{i+1}| > \varepsilon h_i$, then y_{i+1} is rejected and the integration step is repeated using αh_i as the new stepsize

- (b) if $|\ddot{y}_{i+1} - y_{i+1}| \leq \varepsilon h_i$, then y_{i+1} is accepted and αh_i is used as the initial estimate for the stepsize h_{i+1} for the next integration step

To reduce the number of rejected steps, Eq. (3.28) is modified to make α more conservative. For the Runge-Kutta-Fehlberg method of order 4, α is taken as

$$\alpha = \sqrt[4]{\frac{\varepsilon h_i}{2|\ddot{y}_{i+1} - y_{i+1}|}} = 0.84 \sqrt[4]{\frac{\varepsilon h_i}{|\ddot{y}_{i+1} - y_{i+1}|}} \quad (3.29)$$

Furthermore, to prevent α from becoming too small or too large, the heuristic is applied:

- (a) if $\alpha \leq 0.1$, set $\alpha = 0.1$
 (b) if $\alpha \geq 4.0$, set $\alpha = 4.0$
 (c) otherwise, use α as computed (3.30)

The 4th order R-K-F method with adaptive stepsize control is implemented thus:

1. Calculate y_{i+1} and $\ddot{y}_{i+1}$ from Eqs. (3.23).
2. Find α from (3.29) and (3.30).
3. If $|\ddot{y}_{i+1} - y_{i+1}| > \varepsilon h_i$, reject y_{i+1} and repeat the integration step using αh_i as the new stepsize; otherwise, accept y_{i+1} and use αh_i as the initial stepsize for the next integration step.

3.8.2 EASY Implementation of the Runge-Kutta-Fehlberg Method

procedure RKF($\varepsilon, xi, yi, xout, yout, h$)

//Applies the Runge-Kutta-Fehlberg method for the ODE $dy/dx = f(x, y)$ on the interval $[xi, xout]$ starting from the point (xi, yi) with initial stepsize h and error tolerance ε . Upon exit, $yout$ is the estimate of $y(x)$ at $xout$ and h is the next stepsize to use.//

while $xi < xout$ do

if $xi + h > xout$ then $h \leftarrow xout - xi$

$\delta_1 \leftarrow h * F(xi, yi)$

$\delta_2 \leftarrow h * F(xi + h/4, yi + \delta_1/4)$

$\delta_3 \leftarrow h * F(xi + 3 * h/8, yi + 3 * \delta_1/32 + 9 * \delta_2/32)$

$\delta_4 \leftarrow h * F(xi + 12 * h/13, yi + 1932 * \delta_1/2197 - 7200 * \delta_2/2197 + 7296 * \delta_3/2197)$

$\delta_5 \leftarrow h * F(xi + h, yi + 439 * \delta_1/216 - 8 * \delta_2 + 3680 * \delta_3/513 - 845 * \delta_4/4104)$

$\delta_6 \leftarrow h * F(xi + h/2, yi - 8 * \delta_1/27 + 2 * \delta_2 - 3544 * \delta_3/2565 + 1859 * \delta_4/4104 - 11 * \delta_5/40)$

$y \leftarrow yi + 25 * \delta_1/216 + 1408 * \delta_3/2565 + 2197 * \delta_4/4104 - \delta_5/5$

$\ddot{y} \leftarrow yi + 16 * \delta_1/135 + 6656 * \delta_3/12825 + 28561 * \delta_4/56430 - 9 * \delta_5/50 + 2 * \delta_6/55$

//See if integration step is OK//

$r \leftarrow |\ddot{y} - y|/h$

$\alpha \leftarrow 0.84 * (\varepsilon/r)^{0.25}$

//Find new h for failed step or for next step//

if $r < \varepsilon$ then $[xi \leftarrow xi + h; yi \leftarrow y]$

case

: $\alpha \leq 0.1$: $h \leftarrow 0.1 * h$

: $\alpha \geq 4.0$: $h \leftarrow 4.0 * h$

: else : $h \leftarrow \alpha * h$

end case

end while

$yout \leftarrow y$

end procedure

(a) Tolerance $\varepsilon = 1.0D - 06$

t	m	x	y	dx/dt	dy/dt
0.000000		1.200000	0.000000	0.000000	-1.049358
0.619217	9	0.903563	-0.517369	-0.844409	-0.461205
1.238434	12	0.316989	-0.392490	-0.929426	1.093599
1.857651	249	-0.430888	-0.587672	-0.897355	-0.629575
2.476868	10	-0.999889	-0.556156	-0.786242	0.602015
3.096085	6	-1.262453	0.000000	0.000001	1.049557
3.715302	6	-0.999888	0.556155	0.786244	0.602011
4.334518	9	-0.430886	0.587668	0.897354	-0.629584
4.953735	249	0.316992	0.392496	0.929422	1.093582
5.572952	15	0.903564	0.517370	0.844405	-0.461208
6.192169	14	1.199997	0.000001	-0.000006	-1.049354
6.811386	9	0.903557	-0.517363	-0.844412	-0.461190
7.430603	12	0.316981	-0.392469	-0.929438	1.093651
8.049820	249	-0.430895	-0.587686	-0.897354	-0.629545
8.669037	10	-0.999893	-0.556158	-0.786234	0.602024
9.288254	6	-1.262451	-0.000001	0.000011	1.049554
9.907471	6	-0.999881	0.556148	0.786248	0.601995
10.526688	9	-0.430879	0.587646	0.897352	-0.629625
11.145905	249	0.317005	0.392524	0.929403	1.093508
11.765121	15	0.903571	0.517377	0.844394	-0.461225
12.384338	14	1.199997	0.000004	-0.000018	-1.049352

It took only 1158 integration steps of the 4th order Runge-Kutta-Fehlberg method to generate this solution table. Compare this to the 20,000 steps it took the classical 4th order Runge-Kutta to generate the same table (in which we used $1/10,000$ of the period as h). It is clear that stipulating an error tolerance and allowing the method to adjust the step size accordingly can lead to substantial savings in computational effort.

(b) Tolerance $\varepsilon = 1.0D - 07$

t	m	x	y	dx/dt	dy/dt
0.000000		1.200000	0.000000	0.000000	-1.049358
0.619217	15	0.903563	-0.517369	-0.844409	-0.461205
1.238434	22	0.316989	-0.392490	-0.929426	1.093599
1.857651	443	-0.430888	-0.587673	-0.897355	-0.629575
2.476868	17	-0.999890	-0.556157	-0.786244	0.602015
3.096085	11	-1.262454	0.000000	0.000000	1.049559
3.715302	11	-0.999890	0.556157	0.786244	0.602015
4.334518	15	-0.430888	0.587673	0.897355	-0.629576
4.953735	442	0.316989	0.392490	0.929426	1.093599
5.572952	25	0.903563	0.517369	0.844409	-0.461205
6.192169	19	1.200000	0.000000	0.000000	-1.049357
6.811386	14	0.903563	-0.517369	-0.844409	-0.461204
7.430603	22	0.316989	-0.392489	-0.929427	1.093602
8.049820	444	-0.430888	-0.587674	-0.897355	-0.629574
8.669037	17	-0.999890	-0.556157	-0.786243	0.602016
9.288254	11	-1.262454	0.000000	0.000001	1.049559
9.907471	11	-0.999890	0.556156	0.786244	0.602014
10.526688	15	-0.430887	0.587672	0.897355	-0.629578
11.145905	442	0.316990	0.392492	0.929425	1.093595
11.765121	25	0.903564	0.517369	0.844408	-0.461206
12.384338	19	1.200000	0.000000	-0.000001	-1.049357

This time, Runge-Kutta-Fehlberg took 2040 integration steps to generate the table. In contrast, it took the classical 4th order Runge-Kutta method 50,000 steps to generate results of comparable accuracy (with $h = 1/25,000$ of the period).

3.9 Hamming's Predictor-Corrector Method

There are many predictor-corrector methods in the numerical literature, e.g., the 4th order Milne method, the 6th order Milne method, the Adams-Moulton method, etc. As a specific example, we will consider Hamming's predictor-corrector method, which is among the more popular in this category. We will omit the detailed derivation of the formulas (see Carnahan, *et al*, pp. 390–391 or James, *et al*, pp. 443–447 for details).

Hamming's corrector equation is

$$y_{i+1} = \frac{9}{8}y_i - \frac{1}{8}y_{i-2} + \frac{3}{8}h(f_{i+1} + 2f_i - f_{i-1}) \quad (3.31)$$

where $f_j \equiv f(x_j, y_j)$. Assuming that y_i, y_{i-2}, f_i and f_{i-1} are exact [i.e., $y_i = y(x_i)$, $y_{i-2} = y(x_{i-2})$, $f_i = f(x_i, y(x_i))$ and $f_{i-1} = f(x_{i-1}, y(x_{i-1}))$], then the local truncation error for (3.31) is

$$e_t = -\frac{1}{40}h^5 f^{(4)}(\xi) \quad x_{i-2} \leq \xi \leq x_{i+1} \quad (3.32)$$

Since $f_{j+1} \equiv f(x_{j+1}, y_{j+1})$, Eq. (3.31) is *implicit* in y_{i+1} and it has to be solved iteratively, say, by successive substitutions. This means that an initial estimate of y_{i+1} , say y_{i+1}^0 , has to be made to get the iterations started. To this end, Hamming uses the 4th order Milne method *predictor equation*

$$y_{i+1}^0 = y_{i-3} + \frac{4}{3}h(2f_i - f_{i-1} + 2f_{i-2}) \quad (3.33)$$

whose local truncation error (assuming again that y_{i-3}, f_i, f_{i-1} and f_{i-2} are exact) is

$$\acute{e}_t = \frac{14}{45}h^5 f^{(4)}(\psi) \quad x_{i-3} \leq \psi \leq x_{i+1} \quad (3.34)$$

Consider now solving the IVP

$$\frac{dy}{dx} = f(x, y) \quad \text{such that} \quad y(x_0) = y_0$$

using Hamming's method. It is obvious that the method cannot be used for the first three integration steps (i.e., for $i = 0, 1, 2$) since the predictor equation requires information at the preceding four data points. Since Hamming's method is 4th order, we can use the 4th order classical Runge-Kutta method to generate y_1, y_2 and y_3 and then continue the integration using the former.

To perform an integration step on the interval $[x_i, x_{i+1}]$, $i \geq 3$, using Hamming's method, we find an initial estimate y_{i+1}^0 using Eq. (3.33) and then 'refine' this estimate by successive substitutions into Eq. (3.31) until some error criterion satisfies a specified tolerance, to yield y_{i+1}^k . Since Eqs. (3.31) and (3.33) are approximate, with truncation errors given by Eqs. (3.32) and (3.34), this basic computational scheme can still be improved.

Using the estimates (3.32) and (3.34) of the truncation errors in the predictor and corrector equations, we have

$$y(x_{i+1}) - y_{i+1}^0 = \frac{14}{45}h^5 f^{(4)}(\psi) \quad (3.35)$$

and

$$y(x_{i+1}) - y_{i+1}^k = -\frac{1}{40}h^5 f^{(4)}(\xi) \quad (3.36)$$

where y_{i+1}^k is the solution value after k iterations of the corrector equation. Dividing (3.35) by (3.36) and assuming that $f^{(4)}(\psi) = f^{(4)}(\xi)$, we get

$$\frac{y(x_{i+1}) - y_{i+1}^0}{y(x_{i+1}) - y_{i+1}^k} = \frac{\frac{14}{45}h^5 f^{(4)}(\psi)}{-\frac{1}{40}h^5 f^{(4)}(\xi)} = -\frac{112}{9}$$

from which

$$\begin{aligned} y(x_{i+1}) - y_{i+1}^0 &= -\frac{112}{9}[y(x_{i+1}) - y_{i+1}^k] \\ y(x_{i+1}) \left(1 + \frac{112}{9}\right) &= y_{i+1}^0 + \frac{112}{9}y_{i+1}^k \end{aligned}$$

or

$$y(x_{i+1}) = \frac{9}{121}y_{i+1}^0 + \frac{112}{121}y_{i+1}^k \quad (3.37)$$

We note that (3.37) gives an approximate solution at x_{i+1} in terms of the initial estimate given by the predictor equation and the last estimate y_{i+1}^k given by the corrector equation. It can be used to give an improved initial estimate $\tilde{y}_{i+1}^0$ and an improved final estimate y_{i+1} .

1. *Improving the initial estimate from Eq. (3.33)*

Subtracting y_{i+1}^0 from both sides of Eq. (3.37), we have

$$\begin{aligned} y(x_{i+1}) - y_{i+1}^0 &= \frac{9}{121}y_{i+1}^0 + \frac{112}{121}y_{i+1}^k - y_{i+1}^0 \\ &= \frac{112}{121}(y_{i+1}^k - y_{i+1}^0) \\ &= \frac{112}{121}(y_i^k - y_i^0) \end{aligned} \quad (3.38)$$

where y_i^0 is the initial estimate given by (3.33) and y_i^k is the last estimate given by (3.31) for the preceding step. The RHS of (3.38) is an estimate of the local truncation error in the initial estimate y_{i+1}^0 given by (3.33). Adding this to y_{i+1}^0 yields an improved initial estimate $\tilde{y}_{i+1}^0$ which accounts for the truncation error inherent in the predictor equation.

$$\tilde{y}_{i+1}^0 = y_{i+1}^0 + \frac{112}{121}(y_i^k - y_i^0) \quad (3.39)$$

This is the value that is initially substituted into the corrector equation to start the iterations.

2. *Improving the last estimate y_{i+1}^k given by (3.31)*

Subtracting y_{i+1}^k from both sides of Eq. (3.37), we have

$$\begin{aligned} y(x_{i+1}) - y_{i+1}^k &= \frac{9}{121}y_{i+1}^0 + \frac{112}{121}y_{i+1}^k - y_{i+1}^k \\ &= -\frac{9}{121}(y_{i+1}^k - y_{i+1}^0) \end{aligned} \quad (3.40)$$

The RHS of (3.40) is an estimate of the local truncation error in the last estimate y_{i+1}^k given by (3.31). Adding this to y_{i+1}^k yields an improved, and final, estimate y_{i+1} which accounts for the truncation error inherent in the corrector equation.

$$y_{i+1} = y_{i+1}^k - \frac{9}{121}(y_{i+1}^k - y_{i+1}^0) \quad (3.41)$$

If the truncation error for the integration step is within allowable limits, we move on to the next integration interval. Otherwise, the stepsize is adjusted, and then the integration process is continued.

3.9.1 Algorithm for Hamming's Predictor-Corrector Method

Given the IVP

$$\frac{dy}{dx} = f(x, y) \quad \text{subject to} \quad y(x_0) = y_0$$

a 4th order Runge-Kutta method is used to generate y_1, y_2 and y_3 . The derivatives f_0, f_1, f_2 and f_3 are then computed from the given ODE. For $i \geq 3$, Hamming's method is applied as follows:

1. Find the initial estimate y_{i+1}^0 using the 4th order Milne predictor equation

$$y_{i+1}^0 = y_{i-3} + \frac{4}{3}h(2f_i - f_{i-1} + 2f_{i-2}) \quad (3.33)$$

2. Find an improved initial estimate $\ddot{y}_{i+1}^0$ which takes into account the truncation error inherent in (3.33) for $i \geq 4$

$$\ddot{y}_{i+1}^0 = y_{i+1}^0 + \frac{112}{121}(y_i^k - y_i^0) \quad (3.39)$$

3. Solve the corrector function iteratively using $\ddot{y}_{i+1}^0$ from (3.39) as initial estimate

$$y_{i+1}^{j+1} = \frac{9}{8}y_i - \frac{1}{8}y_{i-2} + \frac{3}{8}h(f_{i+1}^j + 2f_i - f_{i-1}) \quad (3.31)$$

where $f_{i+1}^j \equiv f(x_{i+1}, y_{i+1}^j)$. [Note: In practice, Eq. (3.31) is applied only once; the step size is chosen to ensure that convergence effectively takes place in one iteration.]

4. Find an estimate of the truncation error e_t from the value y_{i+1}^0 from step 1 and y_{i+1}^k from step 3

$$e_t = -\frac{9}{121}(y_{i+1}^k - y_{i+1}^0)$$

If the value of e_t is not within the prescribed limit, take corrective action by adjusting the stepsize; otherwise

5. Compute the final estimate as

$$y_{i+1} = y_{i+1}^k + e_t \quad (3.41)$$

Hamming's 4th order predictor-corrector method was transcribed into a FORTRAN 90 program and run in double precision for two values of h for the orbiting satellite problem. The total number of function calls made by the procedure was also recorded. Compare these results with those for RK4 and RKF.

(a) $h = 6.19216933/10000; \varepsilon = 10^{-6}$

t	x	y	dx/dt	dy/dt
0.000000	1.2000000	0.0000000	0.0000000	-1.0493575
0.619217	0.9035630	-0.5173691	-0.8444093	-0.4612049
1.238434	0.3169890	-0.3924903	-0.9294265	1.0935996
1.857651	-0.4308582	-0.5876591	-0.8972469	-0.6295357
2.476868	-0.9997625	-0.5561438	-0.7860409	0.6019480
3.096085	-1.2621896	-0.0000966	0.0002200	1.0492523
3.715302	-0.9995217	0.5557614	0.7863306	0.6013257
4.334518	-0.4305318	0.5865987	0.8972957	-0.6314347
4.953735	0.3174956	0.3939411	0.9282290	1.0900531
5.572952	0.9036810	0.5178436	0.8437116	-0.4618736
6.192169	1.1996283	0.0004169	-0.0008333	-1.0489681
6.811386	0.9026346	-0.5161837	-0.8450543	-0.4589897
7.430603	0.3156739	-0.3887555	-0.9317699	1.1024321
8.049820	-0.4318543	-0.5900584	-0.8970272	-0.6246437
8.669037	-1.0003589	-0.5568061	-0.7849074	0.6034312
9.288254	-1.2619426	-0.0004462	0.0016364	1.0488077
9.907471	-0.9985733	0.5545650	0.7870337	0.5990216
10.526688	-0.4294861	0.5831041	0.8971595	-0.6376269
11.145905	0.3193622	0.3983697	0.9251883	1.0787927
11.765121	0.9047155	0.5190901	0.8422142	-0.4643339
12.384338	1.1995996	0.0010627	-0.0024256	-1.0486644

A total of 20,000 integration steps were performed to generate this table with a total of 281,444 function calls were made by Hamming. Compare this with 320,000 function calls made by procedure RK4 to produce a similar table.

(b) $h = 6.19216933/25000; \varepsilon = 10^{-6}$

t	x	y	dx/dt	dy/dt
0.000000	1.2000000	0.0000000	0.0000000	-1.0493575
0.619217	0.9035631	-0.5173691	-0.8444093	-0.4612049
1.238434	0.3169891	-0.3924903	-0.9294265	1.0935995
1.857651	-0.4308875	-0.5876728	-0.8973537	-0.6295748
2.476868	-0.9998880	-0.5561563	-0.7862403	0.6020142
3.096085	-1.2624499	-0.0000014	0.0000038	1.0495543
3.715302	-0.9998839	0.5561502	0.7862452	0.6020037
4.334518	-0.4308819	0.5876554	0.8973543	-0.6296065
4.953735	0.3169976	0.3925143	0.9294069	1.0935402
5.572952	0.9035652	0.5173767	0.8443976	-0.4612163
6.192169	1.1999939	0.0000066	0.0000142	-1.0493511
6.811386	0.9035476	-0.5173497	-0.8444200	-0.4611680
7.430603	0.3169673	-0.3924287	-0.9294643	1.0937452
8.049820	-0.4309042	-0.5877126	-0.8973501	-0.6294932
8.669037	-0.9998981	-0.5561670	-0.7862213	0.6020390
9.288254	-1.2624458	-0.0000070	0.0000276	1.0495469
9.907471	-0.9998680	0.5561305	0.7862571	0.6019652
10.526688	-0.4308644	0.5875974	0.8973515	-0.6297094
11.145905	0.3170292	0.3925886	0.9293545	1.0933498
11.765121	0.9035827	0.5173974	0.8443724	-0.4612577
12.384338	1.1999934	0.0000171	-0.0000412	-1.0493460

A total of 50,000 integration steps were performed to generate this table with a total of 624,296

function calls were made by Hamming. Compare this with 800,000 function calls made by procedure RK4 to produce a similar table.